LifeLink Pro

LifeLink Pro

Complete System Documentation & Client Handover Guide

Project documentation · Technical documentation · User manual · Administrator manual

Document information

Project	**LifeLink Pro**
Document type	**Complete System Documentation**
Version	**1.0**
Date	**26 October 2026**
Database	**MySQL 8**
Backend	**Node.js + Express 5**
Frontend	**React 19 (Vite 8)**
Status	**Production readiness review of the current implementation**

Every statement in this document was checked against the project source code, database scripts and test results.
Features that are not finished are labelled Partially Implemented, Future Enhancement or Not verified.

Table of Contents

About this document . . . 4

Part A — Overview . . . 6

1. Executive Summary . . . 7
2. Project Overview . . . 8
3. Problem Statement . . . 9
4. Project Objectives . . . 9
5. System Scope . . . 10
6. Target Users . . . 11
7. User Roles & Permissions . . . 11

Part B — Using LifeLink Pro . . . 13

8. Complete User Workflow . . . 14
9. Customer / End-User Manual . . . 15
10. Administrator Manual . . . 25
11. Feature Reference (module by module) . . . 30

Part C — Technical Documentation . . . 34

12. System Architecture . . . 35
13. Technology Stack . . . 36
14. Frontend Architecture . . . 37
15. Backend Architecture . . . 39
16. Database Architecture . . . 41
17. Database Tables . . . 42
18. Authentication & Authorization . . . 46
19. API Documentation . . . 48
20. HTTP Status Codes . . . 56

Part D — Setup, Deployment & Operations . . . 57

21. Installation Guide . . . 58
22. MySQL Installation & Database Setup . . . 59
23. Environment Configuration . . . 59
24. Database Migration & Seed . . . 60
25. Running the Application . . . 62
26. Deployment Guide . . . 63
27. Production Configuration & Administrator Accounts . . . 65
28. Security . . . 66

29. Monitoring & Maintenance . . . 67
30. Troubleshooting . . . 68
31. Backup & Recovery . . . 69

Part E — Quality, Limitations & Handover . . . 70

32. Testing & Validation . . . 71
33. Security Checklist . . . 72
34. Known Limitations . . . 72
35. Future Enhancements . . . 74
36. Project File Structure . . . 75
37. Developer Handover Guide . . . 76
38. Migration History (MongoDB → MySQL) . . . 77
39. Support & Maintenance . . . 78
40. Client Handover Checklist . . . 78
41. Glossary . . . 79
42. Final System Summary . . . 80

About this document

This is the official handover documentation for **LifeLink Pro**, a community emergency and social-service web platform. It is written for three audiences and each part says who it is for:

Reader	Questions answered	Start with
Customers and end users	How do I use LifeLink Pro?	Part B, sections 9 and 11
Administrators / operators	How do I operate and manage LifeLink Pro?	Part B, section 10; Part D, sections 28–31
Management, project managers, clients	What was built, what works, what is missing?	Part A, sections 1–7; Part E, sections 32–35, 40 and 42
Developers, deployment and support teams	How do I install, configure, deploy, maintain and extend it?	Parts C and D, section 37

Document control

Project	LifeLink Pro
Document type	Complete System Documentation & Client Handover Guide
Version	1.0
Date	26 October 2026
Application version	server/package.json and client/package.json do not carry a release number (server 1.0.0, client 0.0.0 are npm defaults). This document describes the source code as inspected on 26 October 2026.
Stack	React 19 + Vite 8 (frontend) · Node.js + Express 5 (backend) · MySQL 8 (database)
Status	Production readiness review of the current implementation. The application runs and passes its automated checks; items listed in section 34 must be addressed before a public launch.
Confidentiality	Contains no passwords, secrets or API keys. Configuration values are shown as placeholders only.

How statements are labelled

Everything in this document was checked against the source code (client/, server/, database/), the SQL scripts and test results. Feature status uses these labels:

Label	Meaning
Implemented	Works end to end in the current code (user interface, API and database).
Partially Implemented	Some parts exist (for example a database column or an API endpoint) but the feature is not complete. The limitation is stated next to it.
Future Enhancement	Not in the current code. A reasonable next step, described so it is not mistaken for an existing feature.
Not verified	Could not be confirmed from the code or a test (for example, anything about a future production server).

Screenshots

The project repository contains no screenshots. The 16 screenshots in this document were captured on 26 October 2026 from the application running locally with the **demo seed data** (database/seeds/001_initial_seed.sql) at a 1024-pixel-wide browser window. Names such as “Demo Donor A” are demo records, not real people. No screenshot was edited or mocked up.

PART A

Overview

For clients, management and everyone new to the project.

1. Executive Summary
2. Project Overview
3. Problem Statement
4. Project Objectives
5. System Scope
6. Target Users
7. User Roles & Permissions

1. Executive Summary

LifeLink Pro is a web application that brings community help services together in one place. People can ask for emergency help, offer blood or food, sign up as volunteers, find hospitals and ambulances, and take part in community events. A small team of administrators can see everything that is submitted and remove records that are no longer needed.

1.1 What problem it solves

In many communities, information about blood donors, surplus food, volunteers, hospitals and ambulances is scattered across phone lists, messaging groups and paper registers. When help is needed quickly, nobody knows where to look. LifeLink Pro gives these services one shared, searchable home and one simple form for each kind of request or offer.

1.2 Major modules

Module	What people can do
Emergency SOS	Send an urgent request (type, name, phone, location, description). Coordinators see it in the admin area. It is **not** an automatic dispatch service; the app always tells people to call 112 or 108 first.
Blood donation	Join a blood donor registry (blood group, age 18–65, phone, location).
Food donation	Offer surplus food for pickup (type, quantity, pickup location, contact).
Volunteers	Register as a volunteer with skills, location and availability.
Community events	Browse upcoming and past events, join an event, or create a new one.
Hospitals	Search a directory of hospitals and add a hospital to it.
Ambulances	Search a directory of ambulances (available / busy) and register an ambulance.
Accounts	Create an account, sign in, use a personal dashboard and profile page.
Administration	Separate admin sign-in, a dashboard with live totals, and eight management screens to search, filter and delete records.

1.3 How it helps users and administrators

- Anyone can use every service **without creating an account**; an account adds a personal dashboard.
- Each form explains what is needed, checks the answers before sending, and confirms what was saved.
- Administrators see live totals and every submitted record, can search and filter them, and can delete records after a confirmation step.

> Current status in one sentence
>
> The application is functionally complete for registering, listing and deleting records, passes its automated checks (78/78 API checks and 71/71 security checks on 26 October 2026), and needs the production items in section 34 (for example rate limiting and dependency updates) before a public launch.

2. Project Overview

LifeLink Pro has three parts that work together:

Component	Folder	Role
Web application (frontend)	`client/`	React single-page application. Everything people see and click: pages, forms, the admin area.
REST API (backend)	`server/`	Node.js + Express server. Receives requests from the web app, checks them, reads and writes the database, and returns JSON.
Database	`database/`	MySQL 8 database named `lifepro`. The schema and demo data are plain SQL files, applied by two npm scripts.

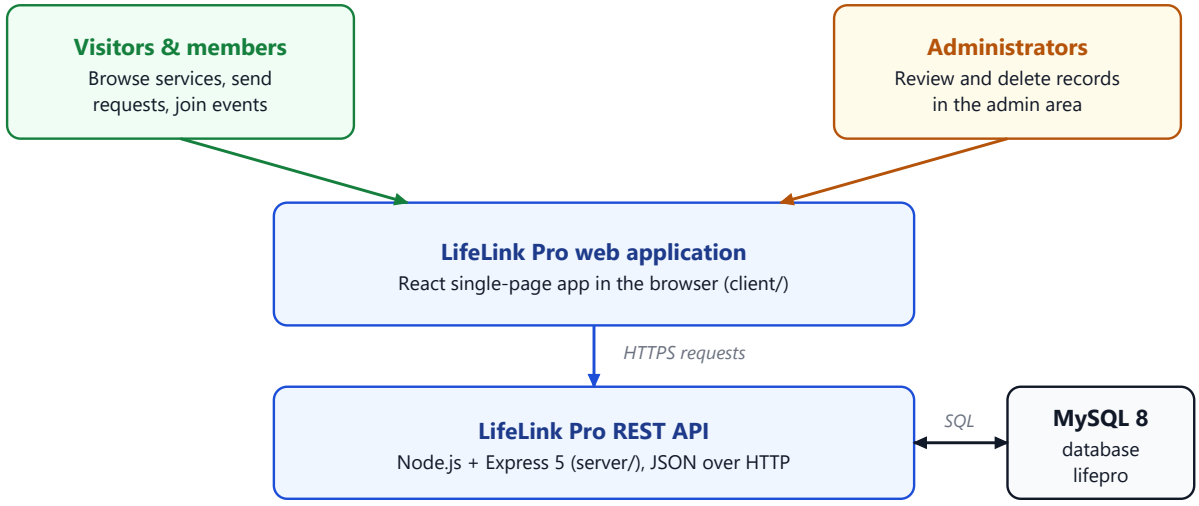


Figure 1 — LifeLink Pro at a glance

2.1 System capabilities

- Public website with 16 public pages (home, help, seven services, three directories with their registration forms, sign-in, account creation, administrator sign-in, page-not-found).
- Member area: dashboard and profile (sign-in required).
- Admin area: dashboard and eight management screens (administrator account required).
- 35 API endpoints (section 19), protected admin endpoints using JSON Web Tokens (JWT).
- Eight business tables plus a migration bookkeeping table in MySQL.
- Responsive layout checked from 320-pixel phones to 1920-pixel desktop screens.

2.2 Administrator capabilities

Administrators can view all records of every type, search and filter them, and delete emergency requests, blood donors, food donations, volunteers, events, hospitals and user accounts. Ambulances can be viewed but not deleted (the API has no delete endpoint for them). Administrators cannot edit records, change an emergency's status or change a user's role from the application.

3. Problem Statement

Community help depends on people finding each other quickly. Without a shared system:

Area	Typical problem	What LifeLink Pro provides
Blood donation	Donor contacts live in personal phones; matching a blood group takes many calls.	A donor registry with blood group, age, phone and location, visible to administrators by group.
Food donation	Surplus food from events or households is wasted because nobody knows who can collect it.	A pickup form; administrators see what is offered, where, and how to call the donor.
Volunteers	Volunteer details are scattered and hard to search.	A volunteer register with skills, location and availability, filterable by availability.
Emergency requests	Requests for help arrive through many channels and get lost.	One SOS form whose requests appear, newest first, in the admin area. It complements — and never replaces — calling emergency numbers.
Hospitals & ambulances	People do not know which hospitals accept emergencies or which ambulances exist.	Public directories with search, filters and one-tap call buttons.
Events	Drives and camps are announced informally; organisers cannot see interest.	A public event list with a Join button that increases the participant count.
Administration	No single place to review or clean up submitted information.	An admin dashboard with totals and management screens with delete.

> Scope boundary
>
> LifeLink Pro records and displays information. It does not dispatch ambulances, notify donors, send SMS or e-mail, or track anyone's location. Coordinators act on the information outside the application (for example by phone).

4. Project Objectives

The objectives below are the ones the current implementation actually supports.

Objective	How the implementation meets it	Status
Centralised access to community services	One site with grouped navigation (Get help / Get involved / Events / Help) and a home page linking every service.	**Implemented**
Let people register and sign in	Account creation, sign-in, sign-out, session expiry handling; bcrypt-hashed passwords and 7-day JWT sessions.	**Implemented**
Let people take part in donation activities	Blood donor and food donation forms saved to MySQL.	**Implemented**
Manage volunteers	Volunteer registration; admin list with availability filter and delete.	**Implemented**
Provide emergency information	SOS request form; 112/108 call links on emergency-related pages; hospital and ambulance directories.	**Implemented**
Manage events	Create, list and join events; admin list and delete.	**Implemented**

Objective	How the implementation meets it	Status
Let administrators manage system data	Admin dashboard and management screens with search, filters and delete. No editing.	**Partially Implemented**

5. System Scope

Each area was verified against the routes, controllers, pages and database schema.

Area	Description	Status	Notes
Authentication	Create account, sign in, sign out, session expiry	**Implemented**	No password reset, e-mail verification or profile editing.
Emergency	SOS request submission and admin review	**Implemented**	Status workflow (Pending → Accepted → Resolved) exists only as a database column: **Partially Implemented**.
Blood donation	Donor registry	**Implemented**	One registration per phone number.
Food donation	Food offered for pickup	**Implemented**	The `available` column is always true; nothing marks food as collected.
Volunteers	Volunteer registration	**Implemented**	
Events	List, create, join	**Implemented**	Joining increases a counter; who joined is not stored (remembered per browser only).
Hospitals	Directory and registration	**Implemented**	No editing.
Ambulances	Directory and registration	**Partially Implemented**	No API to update availability or delete an ambulance; admin screen is read-only.
Admin dashboard	Totals and summaries	**Implemented**	Data from GET /api/admin/all.
Admin management	View, search, filter, delete	**Implemented**	Ambulances view-only.
User management	View and delete accounts	**Partially Implemented**	No role change or editing; the “cannot delete yourself” rule is enforced in the UI only.
Help & FAQ	Static help page	**Implemented**	
Notifications (SMS / e-mail)		**Future Enhancement**	Not present.
Real-time dispatch / tracking		**Future Enhancement**	Not present.

Table 1 — Verified system scope

6. Target Users

6.1 Visitors (no account)

Anyone who opens the website. Visitors can use **every public service**: send an emergency request, register as a blood donor, donate food, register as a volunteer, browse, join and create events, and browse and add hospitals and ambulances. No sign-in is required for any of these.

6.2 Members (role “user”)

People who created an account. In addition to everything a visitor can do, members get a personal dashboard (quick links to every service, upcoming events, their account details) and a profile page. Self-registration always creates this role.

6.3 Administrators (role “admin”)

Coordinators who manage the platform. They sign in through a separate administrator sign-in page and use the admin area: dashboard, eight management screens, search, filters and delete. An administrator account cannot be created from the website; the first one comes from the demo seed (development only) or is promoted with an SQL statement (section 27).

> Other roles
>
> The database allows exactly two roles: `ENUM('user', 'admin')`. There are no other roles.

7. User Roles & Permissions

Verified from server/routes (which endpoints use the `protectAdmin` middleware) and client/src/App.jsx (which pages use `RequireAuth`).

Capability	Visitor	Member	Admin	Enforced by
View home, help, events, hospital and ambulance directories	Yes	Yes	Yes	Public pages and API
Send an emergency request	Yes	Yes	Yes	Public API
Register as blood donor / donate food / volunteer	Yes	Yes	Yes	Public API
Create an event, join an event	Yes	Yes	Yes	Public API
Register a hospital or an ambulance	Yes	Yes	Yes	Public API
Create an account	Yes	—	—	Signed-in users are redirected away
Sign in (member sign-in page)	—	Yes	Yes	Admins are sent to the admin dashboard
Personal dashboard and profile	—	Yes	Yes	Browser only (RequireAuth)
Administrator sign-in page	—	Refused	Yes	Browser checks role before saving the session

Capability	Visitor	Member	Admin	Enforced by
Admin dashboard and management screens	—	—	Yes	RequireAuth role=admin + API protectAdmin
Delete records (7 types)	—	—	Yes	API protectAdmin (JWT + database role check)
Delete own administrator account	—	—	UI: No	Browser only — the API would allow it
Delete or edit ambulances; edit any record	—	—	—	No such endpoint
Raw public list APIs (GET /api/blood etc.)	Yes	Yes	Yes	Public API (see security note)

Table 2 — Permissions matrix

Security note on public endpoints

The list endpoints GET /api/blood, /api/emergency, /api/volunteer and /api/food return names and phone numbers to anyone who calls them, even though the web app never shows these lists publicly. Decide before launch whether to protect or remove them (section 34).

PART B

Using LifeLink Pro

Step-by-step guides for customers and administrators, and a complete reference for every feature.

8. Complete User Workflow
9. Customer / End-User Manual
10. Administrator Manual
11. Feature Reference (module by module)

8. Complete User Workflow

Every public service follows the same pattern: choose a service, fill in a short form, and receive a confirmation. An account is optional.

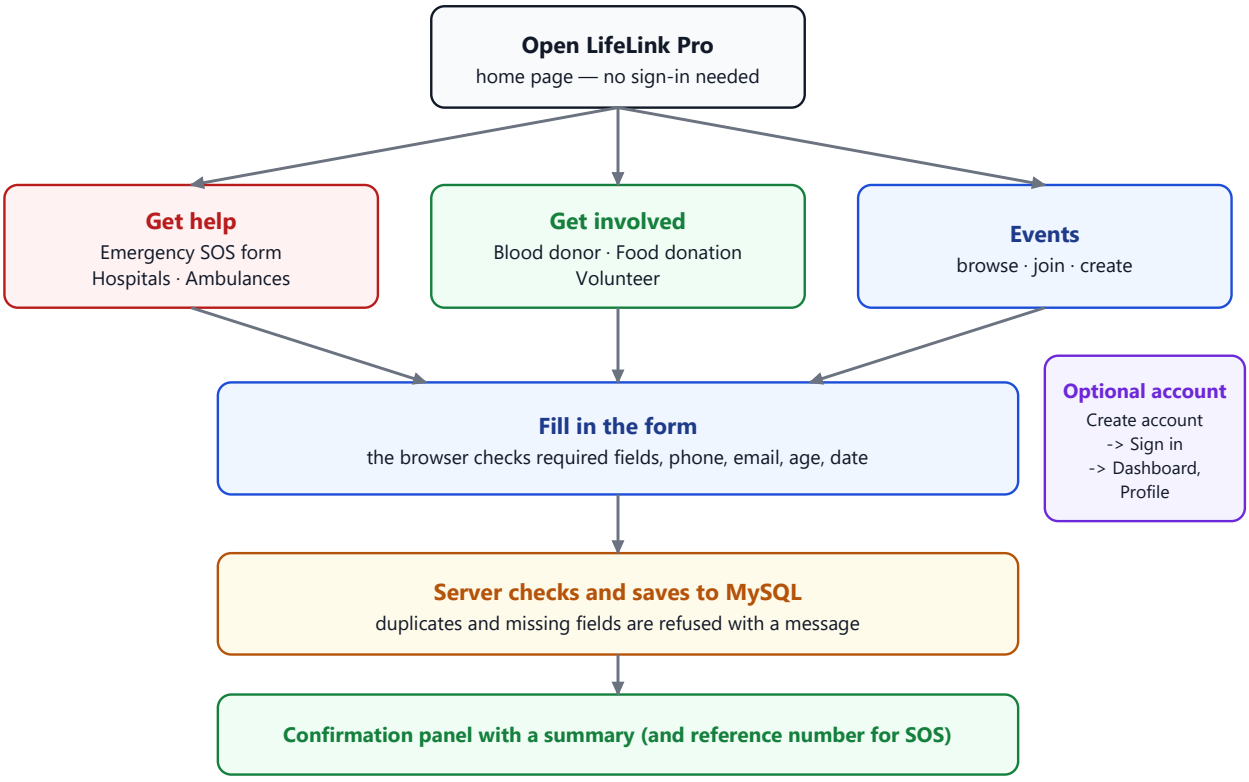


Figure 2 — User workflow

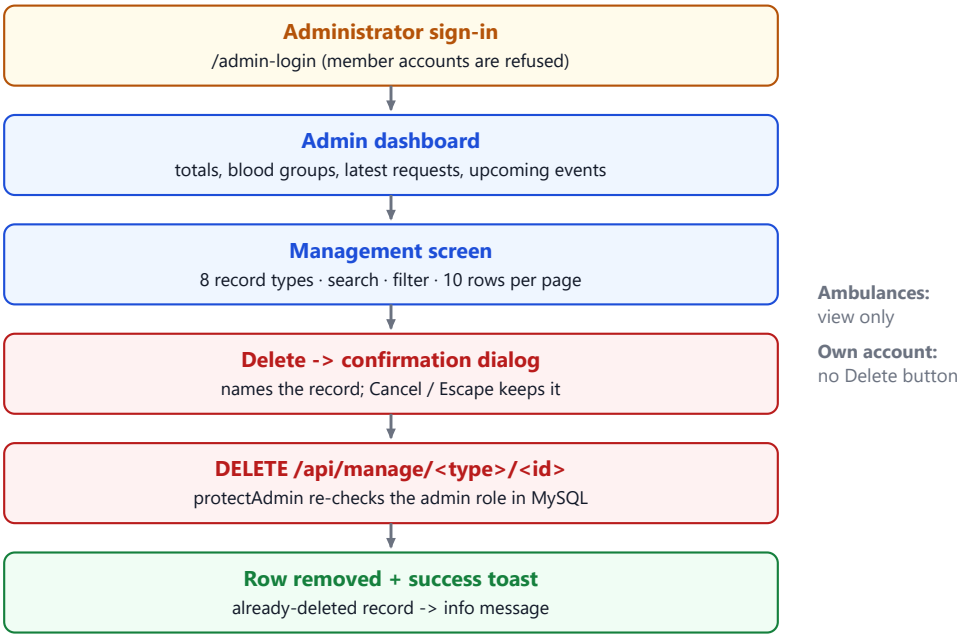


Figure 3 — Administrator workflow

9. Customer / End-User Manual

This manual is written for people who have never used LifeLink Pro. No technical knowledge is needed.

> In a life-threatening emergency, call first
>
> Call **112** (emergency services) or **108** (ambulance) before using the website. LifeLink Pro shares your request with coordinators but does not send help automatically.

9.1 Getting started

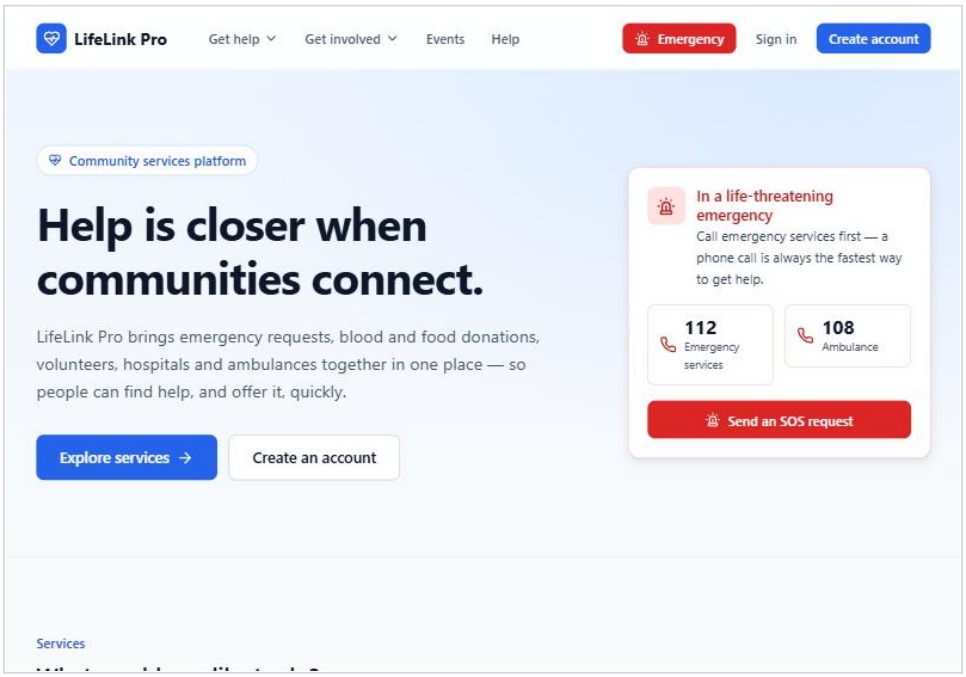


Figure 4 — Home page (public)

1. Open the LifeLink Pro web address your organisation gave you in any modern browser (Chrome, Edge, Firefox or Safari), on a phone, tablet or computer.
2. The home page explains the services. Use **Get help** (emergency, hospitals, ambulances) or **Get involved** (blood, food, volunteering) at the top, or **Events** and **Help**.
3. The red **Emergency** button (shown as **SOS** on small phones) is always at the top of the page.
4. On a phone, tap the menu button (three lines) at the top right to see all links.

9.2 Creating an account (optional)

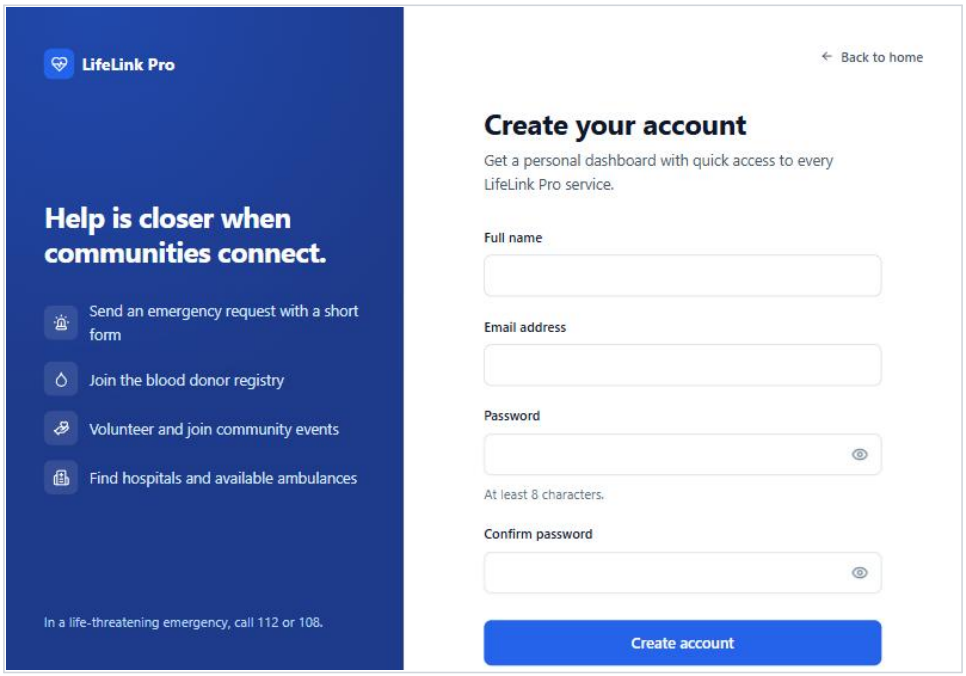


Figure 5 — Create an account

You can use every service without an account. An account gives you a personal dashboard.

1. Select **Create account** at the top right.
2. Enter your **full name**, **email address**, a **password** of at least 8 characters, and the same password again under **Confirm password**. The eye icon shows or hides the password.
3. Select **Create account**. If something is missing or wrong, a red message appears under that field.
4. When it succeeds you are taken to the sign-in page, which says *"Your account is ready"* and already contains your email address.

If the email is already registered, the page says so and offers **Go to sign in**.

9.3 Signing in and out

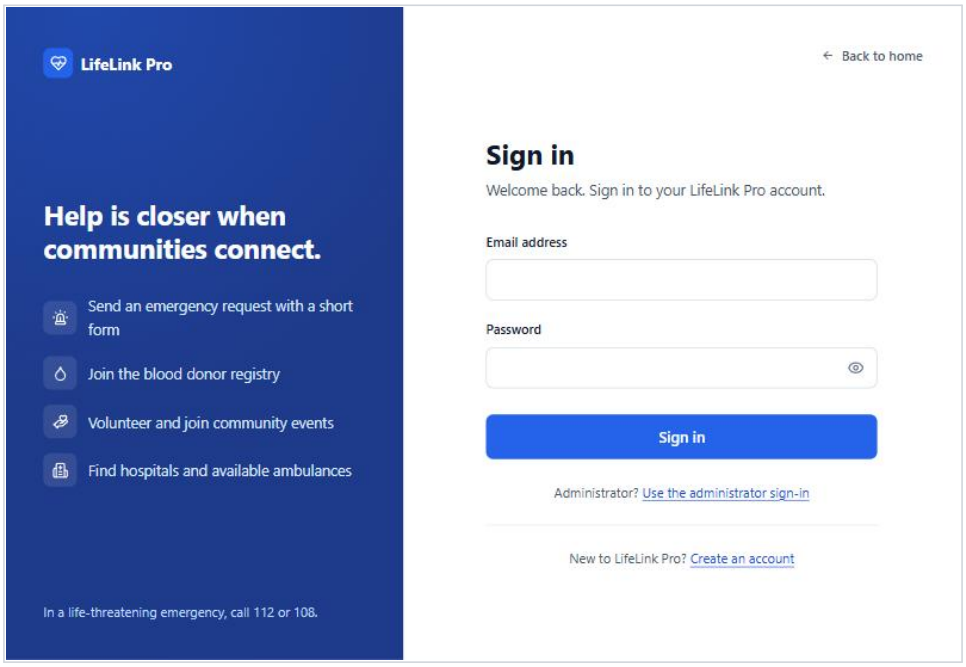


Figure 6 — Sign in

1. Select **Sign in** at the top right.
2. Enter your email address and password, then select **Sign in**.
3. You arrive at **Your dashboard**. If you were sent to the sign-in page from a members-only page, you are returned to that page instead.
4. To sign out, open the menu with your initials at the top right and select **Sign out**. You return to the home page.

What you see	What it means / what to do
“Enter your email address.” / “Enter your password.”	A field is empty. Fill it in.
“Sign-in failed — Invalid email or password”	The email or password is wrong. The message does not say which, for security. Check both and try again.
“Your session has expired”	You were signed in for more than 7 days, or your account was removed. Sign in again.
Sent to the sign-in page	The page you opened is for members only. Sign in first.
“We couldn't reach the LifeLink Pro server…”	Your internet connection or the server is down. Try again later.

9.4 Your dashboard and profile

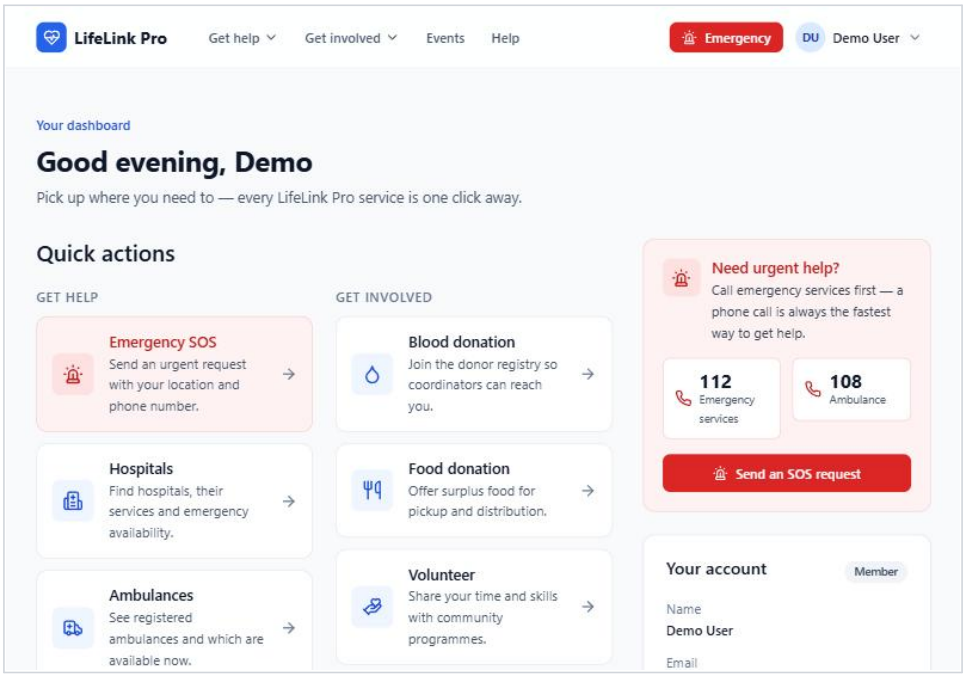


Figure 7 — Member dashboard

After signing in, the dashboard greets you and shows quick links to every service, the upcoming events (with a **Join event** button), emergency numbers, and your account card. **Your profile** (from the menu with your initials) shows your name, email and account type. These details cannot be changed in the app.

9.5 Requesting emergency help (SOS)

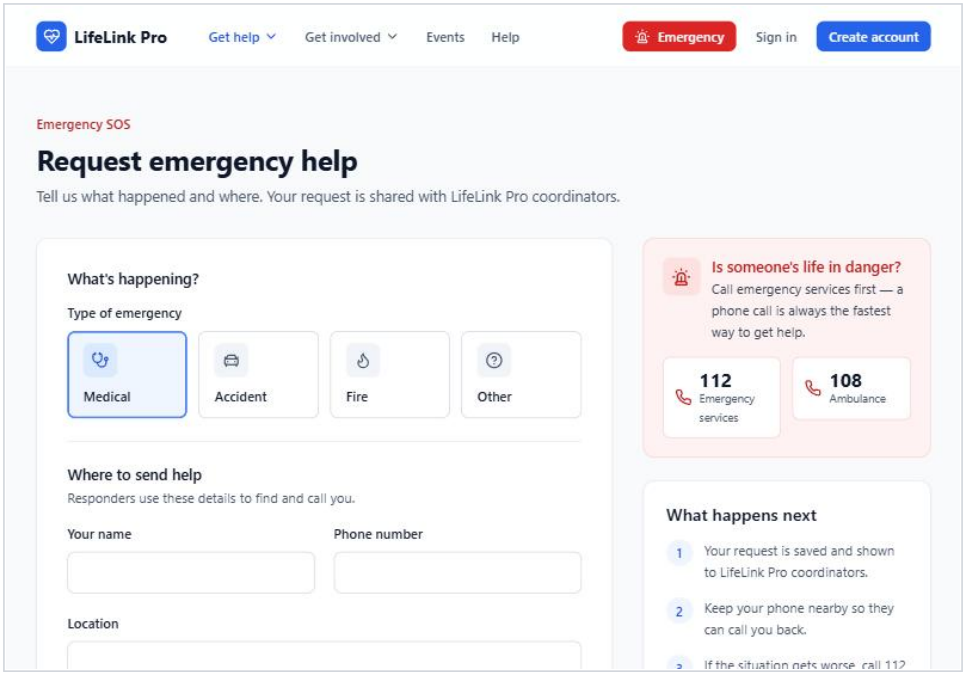


Figure 8 — Emergency SOS request form

1. Select the red **Emergency** button.
2. Choose the **type of emergency**: Medical (selected by default), Accident, Fire or Other.
3. Enter **your name**, a **phone number** responders can call, and the **location** (street, landmark or area).

4. Optionally describe **what happened** (up to 1,000 characters).
5. Select **Send emergency request**.
6. A confirmation shows your **reference number** (for example #15), the type, location and phone. Keep your phone nearby.

The same name, phone and location cannot be sent twice; the page then says *"Emergency request already exists."*

9.6 Registering as a blood donor

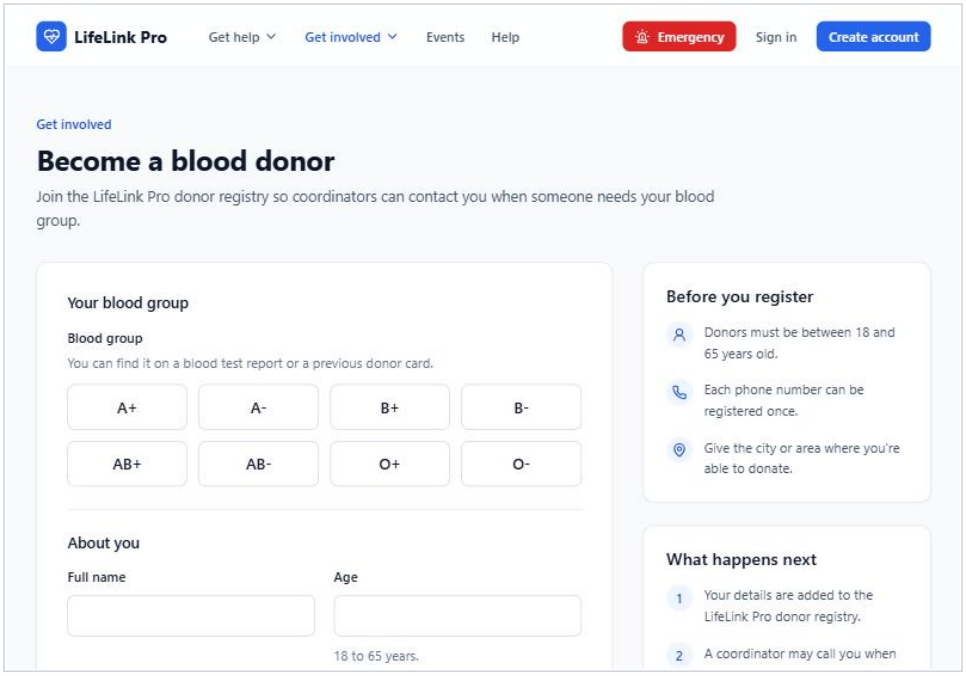


Figure 9 — Blood donor registration

1. Open **Get involved → Blood donation**.
2. Choose your **blood group** (A+, A−, B+, B−, AB+, AB−, O+, O−).
3. Enter your **full name**, **age** (18 to 65), **phone number** and **city or area**.
4. Select **Register as a donor**. The confirmation repeats your blood group, location and phone.

Each phone number can be registered only once. A second attempt shows *"You are already registered as a blood donor."*

9.7 Donating food

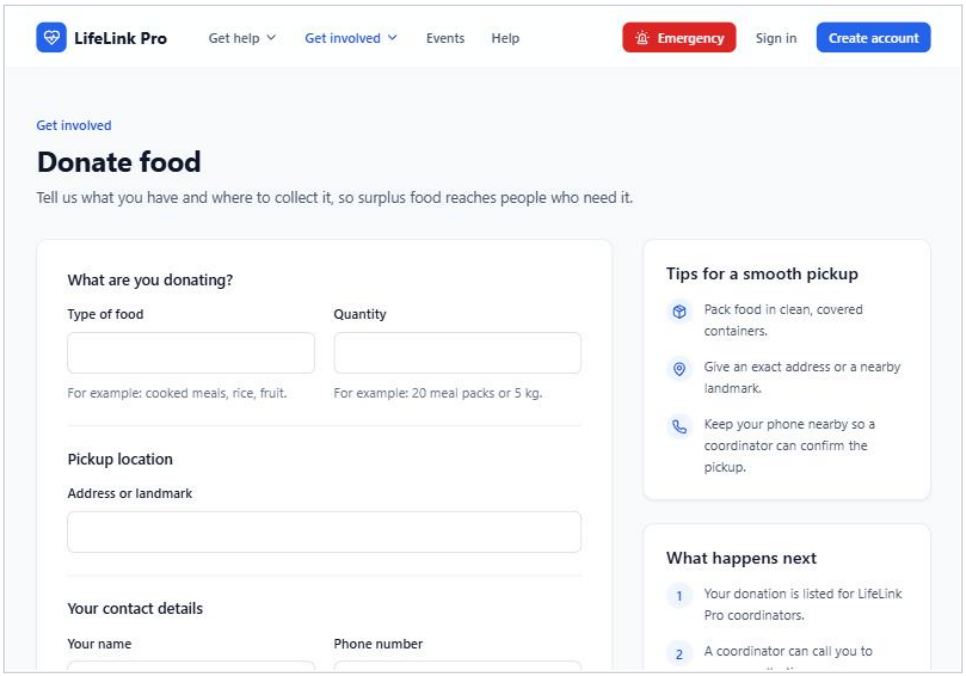


Figure 10 — Food donation form

1. Open **Get involved → Food donation**.
2. Describe the **type of food** (for example cooked meals, rice, fruit) and the **quantity** (for example 20 meal packs or 5 kg).
3. Enter the **pickup address or landmark**, **your name** and **phone number**.
4. Select **Submit donation**. Use **Donate more food** on the confirmation to start a new one.

9.8 Volunteering

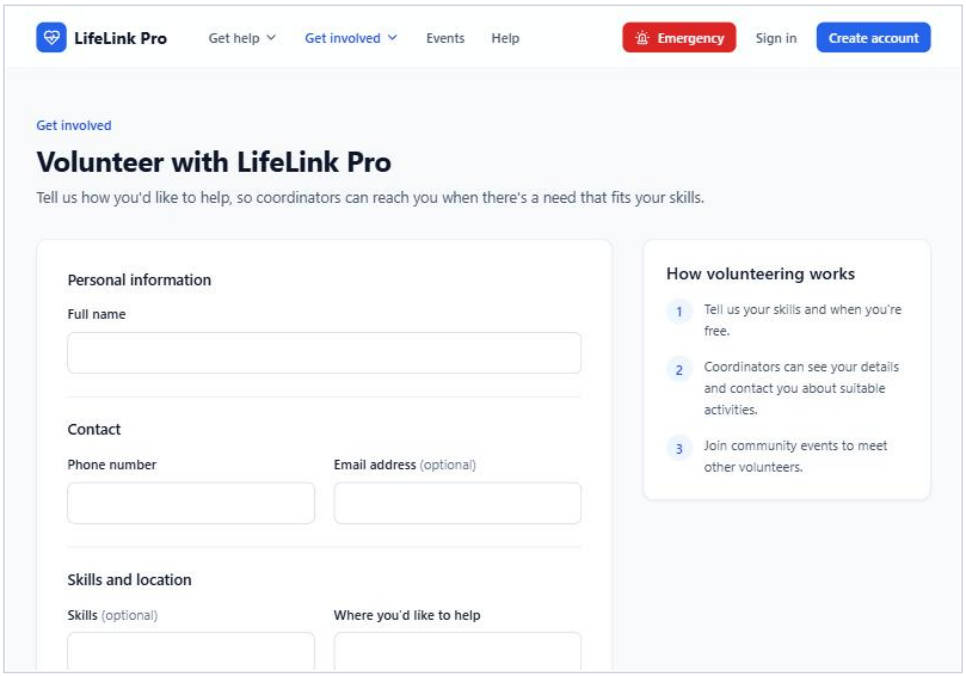


Figure 11 — Volunteer registration

1. Open **Get involved → Volunteer**.
2. Enter your **full name** and **phone number**; **email** and **skills** are optional.
3. Enter **where you'd like to help** (city or area) and choose when you are available: **Full time**, **Part time** or **Weekends**.
4. Select **Register as a volunteer**.

9.9 Community events

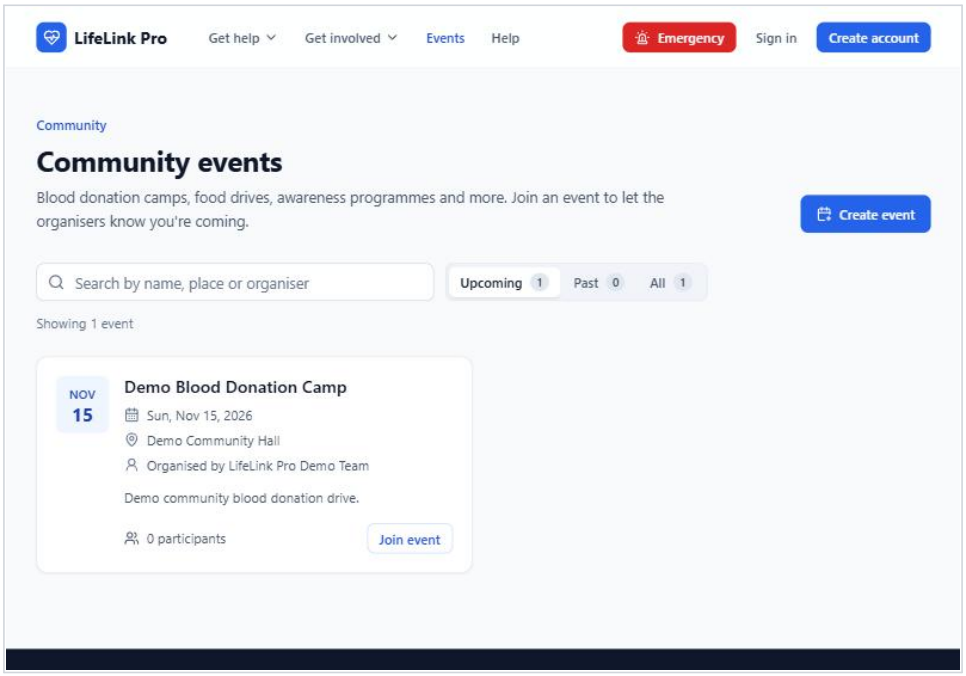


Figure 12 — Community events

Find an event: open **Events**. Use the search box (name, place or organiser) and the **Upcoming / Past / All** buttons.

Join an event: select **Join event** on an upcoming event. A message confirms it, the participant count goes up by one, and the button changes to **You're going**. Past events show **Event ended**.

Create an event: select **Create event**, enter the **event name**, **date** (today or later), **location**, and optionally a **description** and **organiser name**, then select **Create event**. It appears on the events page immediately.

> Good to know
>
> LifeLink Pro remembers that you joined an event on this device and browser only. Joining from another device counts again, and there is no way to leave an event.

9.10 Finding and adding hospitals

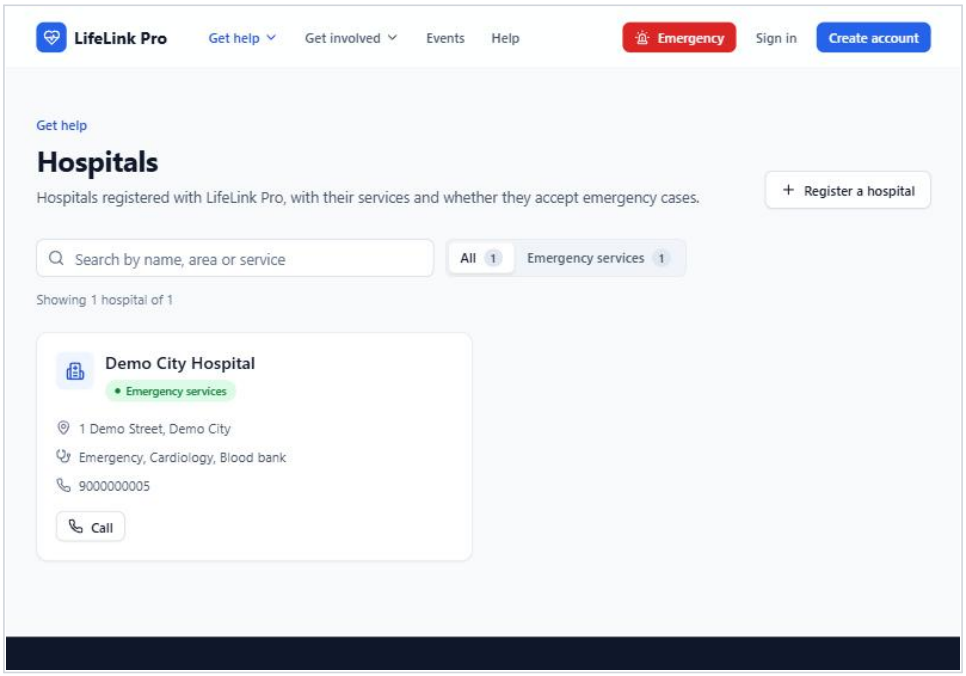


Figure 13 — Hospital directory

Open **Get help** → **Hospitals**. Search by name, area or service, or show only hospitals with **Emergency services**. Each card shows the address, services and phone, with a **Call** button.

To add one, select **Register a hospital** and enter the **hospital name**, **phone number** and **address**; **services** and whether it **accepts emergency cases** are optional. Select **Add hospital**; the hospital appears in the list straight away.

9.11 Finding and registering ambulances

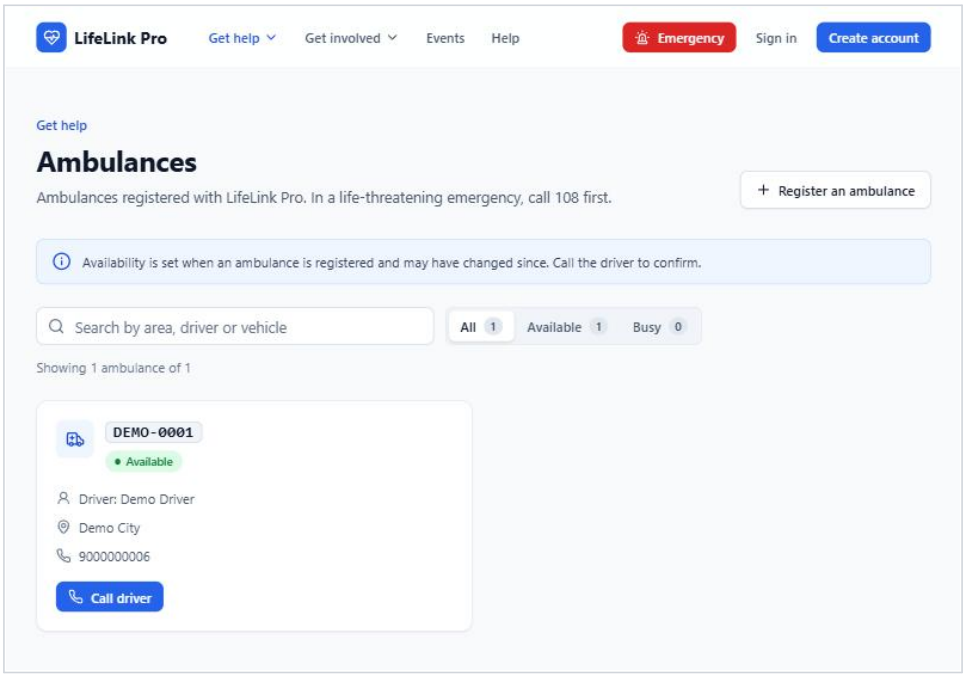


Figure 14 — Ambulance directory

Open **Get help** → **Ambulances**. Search by area, driver or vehicle, or filter by **Available** / **Busy**. Select **Call driver** to phone the driver.

> **Availability may be out of date**
>
> An ambulance's availability is set when it is registered and cannot be changed later. Always call the driver to confirm.

To register one, select **Register an ambulance** and enter the **driver name**, **phone number** (shown publicly), **vehicle number** (converted to capitals), **base location** and **current availability**, then select **Register ambulance**.

9.12 Help page and what to do if something fails

Help at the top of every page answers common questions (emergencies, accounts, blood donation, adding hospitals/ambulances/events, changing your information).

Problem	What to do
A red message appears under a field	Correct that field; the message says exactly what is needed.
"…already registered" or "…already exists"	The same details were sent before. Nothing more is needed.
"We couldn't reach the LifeLink Pro server"	Check your internet connection and try again. If it continues, contact your LifeLink Pro administrator.
"Something went wrong. Please try again."	A temporary server problem. Try again later; your typed information is kept on the page.
A list says "couldn't be loaded"	Select **Try again**.
You need to change or remove your information	Ask a LifeLink Pro administrator. Administrators can delete records; nobody can edit them in the app.

10. Administrator Manual

For coordinators with an administrator account. Administrator accounts cannot be created from the website; ask the technical team (section 27, “Creating the first administrator”).

10.1 Signing in

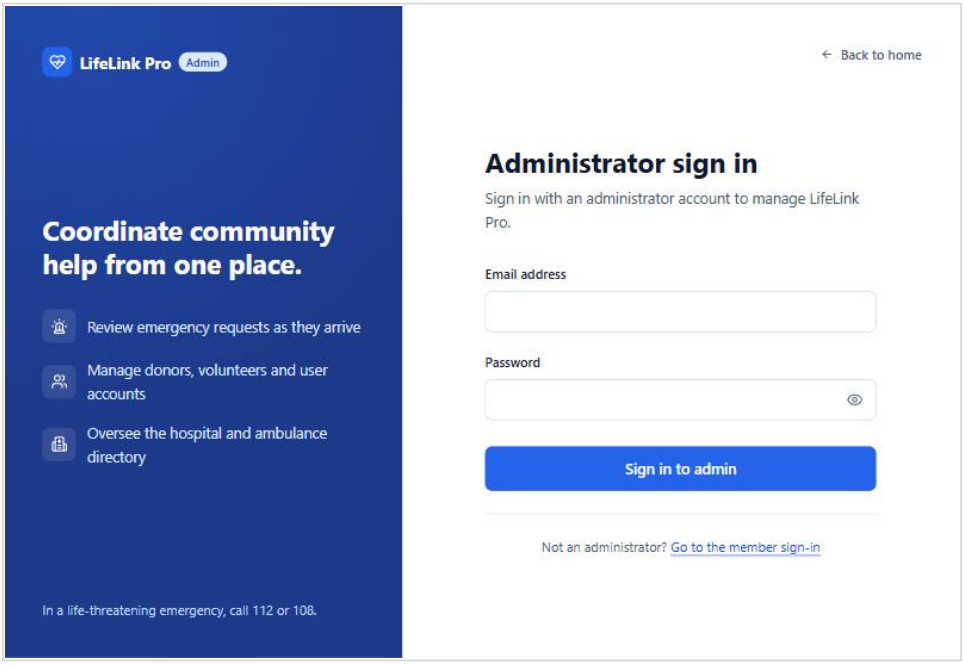


Figure 15 — Administrator sign-in

1. Open the administrator sign-in page: select **Sign in**, then **Use the administrator sign-in** (address ending in `/admin-login`).
2. Enter your administrator email and password and select **Sign in to admin**.
3. You arrive at the **Admin dashboard**. Member accounts are refused with *“This account doesn't have administrator access.”*

Administrators who sign in through the normal sign-in page are also taken to the admin dashboard.

10.2 Dashboard

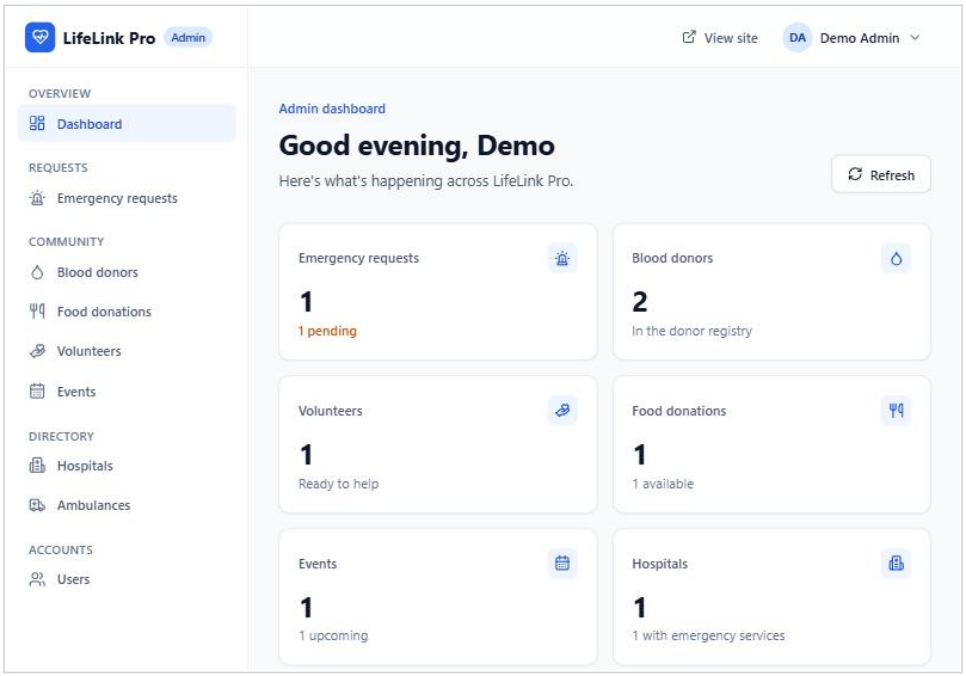


Figure 16 — Admin dashboard

The dashboard shows live totals for emergency requests (with the number pending), blood donors, volunteers, food donations (with the number available), events (with the number upcoming), hospitals (with the number offering emergency services), ambulances (with the number available) and users (with the number of administrators). Below the totals: the latest emergency requests, donors by blood group, and upcoming events. Select **Refresh** to reload. All figures come straight from the database.

10.3 Navigating the admin area

The left sidebar (a menu button on tablets and phones) groups the screens: **Overview** (Dashboard), **Requests** (Emergency requests), **Community** (Blood donors, Food donations, Volunteers, Events), **Directory** (Hospitals, Ambulances) and **Accounts** (Users). **View site** opens the public website.

10.4 Management screens

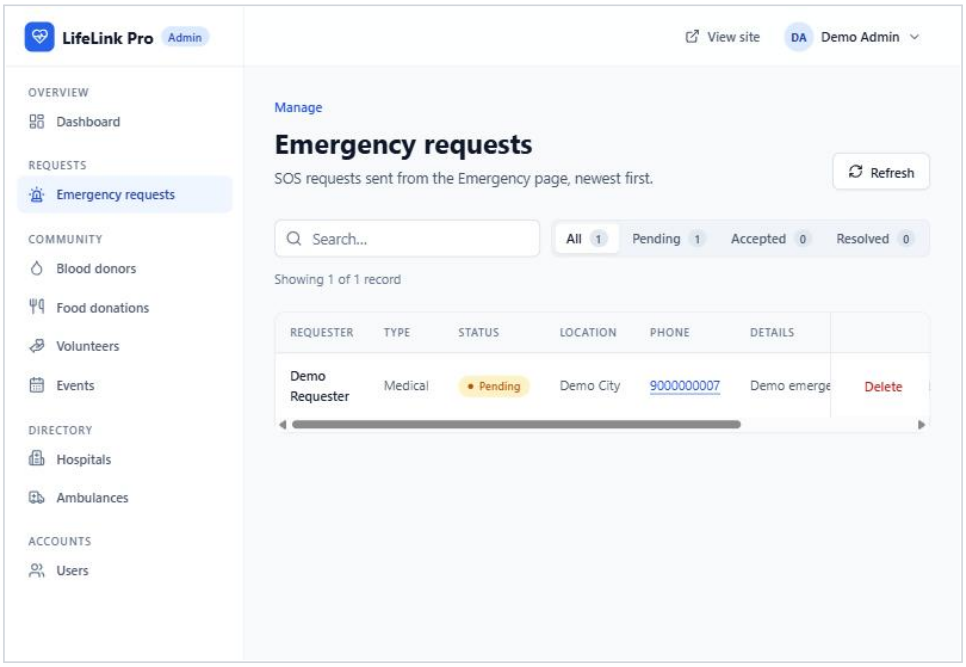


Figure 17 — Emergency requests management screen

Every screen works the same way: a search box, optional filter buttons with counts, a table (cards on phones) with 10 records per page, **Refresh**, and a **Delete** button per record.

Screen	Columns	Filter	Delete
Emergency requests	Requester, Type, Status, Location, Phone, Details, Received	Status: Pending / Accepted / Resolved	Yes
Blood donors	Donor, Blood group, Age, Phone, Location, Added	—	Yes
Food donations	Donor, Food, Quantity, Pickup location, Phone, Status, Added	—	Yes
Volunteers	Name, Phone, Email, Skills, Location, Availability, Added	Availability	Yes
Events	Event, Date, Status (Upcoming / Ended), Location, Organiser, Joined	Upcoming / Past	Yes
Hospitals	Hospital, Phone, Address, Services, Emergency, Added	Emergency services	Yes
Ambulances	Vehicle, Driver, Phone, Base, Status, Added	Available / Busy	No (view only)
Users	Name, Email, Role, Joined	Administrators / Members	Yes, except your own account

Table 3 — Admin management screens

10.5 Deleting a record

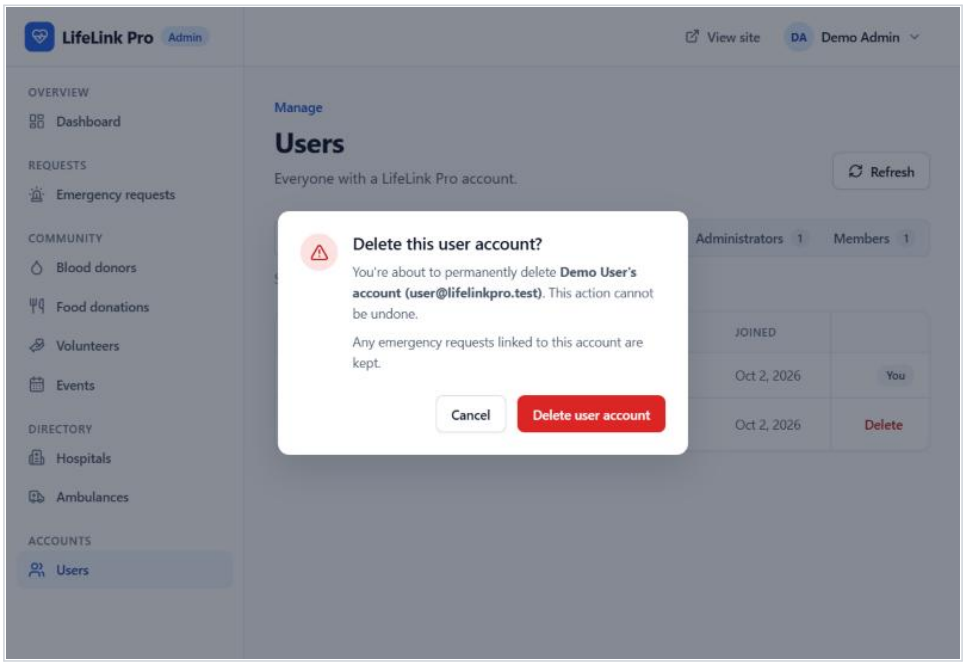


Figure 18 — Delete confirmation dialog

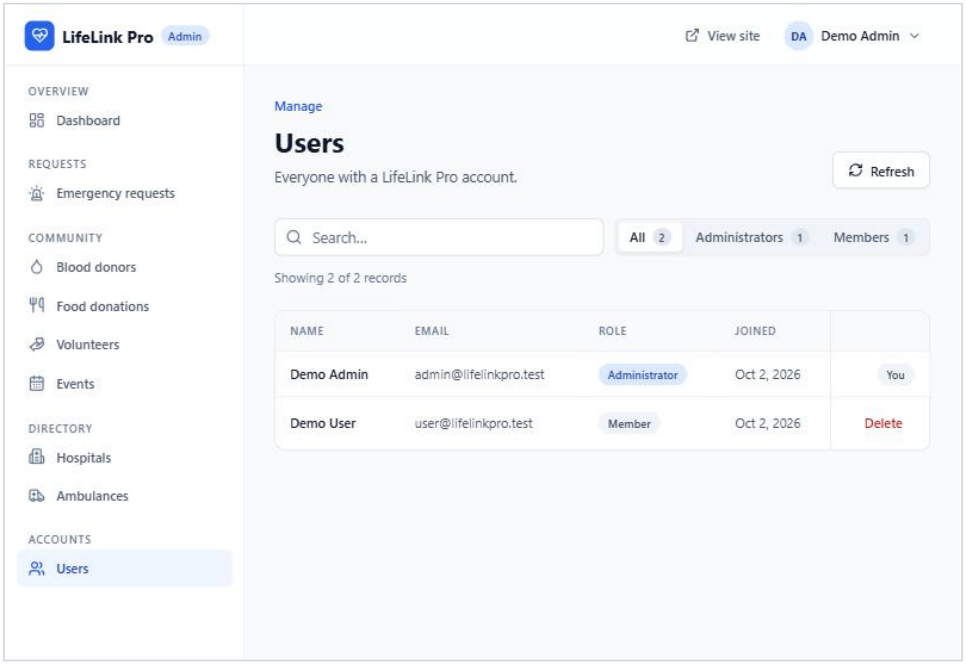


Figure 19 — Users screen: your own account shows "You" instead of Delete

1. Find the record (use search or a filter).
2. Select **Delete** on its row.
3. A dialog names the record and warns that this cannot be undone. Select **Cancel** (or press Escape) to keep it, or the red **Delete …** button to delete it.
4. A message confirms "… deleted." and the row disappears. If another administrator already deleted it, you see "This … had already been deleted."

> **Deleting is permanent**
>
> There is no undo, recycle bin or audit log. Deleting a user keeps any emergency requests linked to that account. Make regular database backups (section 31).

10.6 What administrators cannot do in the app

- Edit any record, or change an emergency request's status.
- Create, promote or demote administrators, or reset passwords.
- Delete or update ambulances.
- Export data or see a history of who deleted what.

10.7 Signing out

Open the menu with your initials (top right) and select **Sign out**. You return to the administrator sign-in page. Sessions also end automatically after 7 days, or as soon as your account is deleted or loses the administrator role.

11. Feature Reference (module by module)

For each feature: purpose, who can use it, access, inputs, validation, result, database effect, API and common errors. “Browser check” means the form refuses to send until fixed; “API check” is enforced by the server for any client.

11.1 Account registration

Purpose	Create a member account for the dashboard and profile.
Who / access	Visitors · `/register` (signed-in users are redirected to the dashboard).
Inputs	Full name, email address, password, confirm password.
Validation	Browser: all required; email format text@text; password 8+ characters; passwords match. API: name non-empty, email and password present, email format; email must be unique.
Result	Redirect to sign-in with a success message and the email filled in. No automatic sign-in.
Database	INSERT into `users` with role 'user' and a bcrypt hash (cost 10).
API	`POST /api/auth/register` → 201.
Common errors	400 “User already exists” (shown as “This email is already registered”); 400 for missing fields or invalid email.

11.2 Sign in / sign out / session

Purpose	Start and end a session.
Who / access	Members and admins · `/login`, `/admin-login`.
Inputs	Email, password.
Validation	Browser: both required, email format. API: credentials checked with bcrypt.
Result	JWT (7 days) and user details stored in the browser (`localStorage` keys `token` and `user`). Members go to /dashboard, admins to /admin/dashboard, or back to the page that required sign-in. Sign-out removes both keys.
Database	Read-only (SELECT user by email).
API	`POST /api/auth/login` → 200.
Common errors	400 “Invalid email or password” (same message for unknown email and wrong password). Any later 401 from the API signs the user out and shows “Your session has expired”.

11.3 Emergency SOS request

Purpose	Share an urgent request with coordinators.
Who / access	Everyone · `/emergency` (red Emergency button).
Inputs	Type (Medical / Accident / Fire / Other), name, phone, location, description (optional).
Validation	Browser: type, name, phone (7–15 digits; digits, spaces, + - ( ) allowed), location. API: name, phone, location required; same name + phone + location rejected.
Result	Confirmation with reference number (database id), type, location, phone.
Database	INSERT into `emergencies`; status defaults to 'Pending'; user_id stays NULL.
API	`POST /api/emergency` → 201.

Common errors	400 "Emergency request already exists."; 400 "Name, phone and location are required."
Limitations	**Partially Implemented**: status never changes from Pending; requests are not linked to the signed-in account; no notifications are sent.

11.4 Blood donor registration

Purpose	Add a donor to the registry.
Who / access	Everyone · `/blood`.
Inputs	Blood group, full name, age, phone, city or area.
Validation	Browser: all required; age a whole number 18–65; phone format. API: name, phone, blood group, location required; phone must not already exist (also a UNIQUE index).
Result	Confirmation with blood group, location and phone.
Database	INSERT into `blood_donors` (available defaults to TRUE).
API	`POST /api/blood/register` → 201.
Common errors	400 "You are already registered as a blood donor".

11.5 Food donation

Purpose	Offer surplus food for collection.
Who / access	Everyone · `/food`.
Inputs	Type of food, quantity (free text), pickup address/landmark, your name, phone.
Validation	Browser: all required; phone format. API: all five required; an identical donation (same name, phone, food type, quantity and location) is rejected.
Result	Confirmation with food, quantity and pickup location; "Donate more food" resets the form.
Database	INSERT into `food_donations` (`name` stores the donor name).
API	`POST /api/food/donate` → 201.
Common errors	400 "This food donation already exists."

11.6 Volunteer registration

Purpose	Register willingness to help.
Who / access	Everyone · `/volunteer`.
Inputs	Full name, phone, email (optional), skills (optional), where you'd like to help, availability (Full Time / Part Time / Weekends).
Validation	Browser: name, phone, location, availability required; email format if given. API: only name is required (other clients may send less).
Result	Confirmation with availability, location and skills.
Database	INSERT into `volunteers` (address column exists but the form does not use it).
API	`POST /api/volunteer/register` → 201.
Common errors	400 "Volunteer name is required." (API only).

11.7 Community events

Purpose	Publish events and let people show they are coming.
Who / access	Everyone · `/events`, `/events/new`; Join also on the member dashboard.
Inputs (create)	Event name, date, location, description (optional), organiser (optional).
Validation	Browser: name, date (today or later), location. API: only the title/eventName is required.
Result	Create: confirmation, event listed immediately. Join: count +1, toast, button shows "You're going" (remembered in the browser key `lifelink.joinedEvents`).
Database	Create: INSERT into `events` (participants 0). Join: `UPDATE events SET participants = participants + 1`.
API	`POST /api/events/create` → 201; `GET /api/events`; `PUT /api/events/join/:id` → 200.
Common errors	404 "Event not found"; 400 "Invalid id."
Limitations	**Partially Implemented**: joining is anonymous and unauthenticated; the API cannot tell who joined, so the count can be raised repeatedly. The `status` column (default 'Active') is not used; Upcoming/Ended is calculated from the date.

11.8 Hospitals

Purpose	Public directory of hospitals.
Who / access	Everyone · `/hospitals`, `/hospitals/register`.
Inputs	Hospital name, phone, address, services (optional), accepts emergency cases (optional: Available / Not Available).
Validation	Browser: name, phone (format), address. API: only the name is required.
Result	Listed immediately with search and an "Emergency services" filter.
Database	INSERT into `hospitals` (the form leaves `location` empty).
API	`POST /api/hospital/register` → 201; `GET /api/hospital`.
Common errors	400 "Hospital name is required." (API only).

11.9 Ambulances

Purpose	Public directory of ambulances with a call button.
Who / access	Everyone · `/ambulance`, `/ambulance/register`.
Inputs	Driver name, phone, vehicle number (upper-cased), base location, availability (Available / Busy).
Validation	Browser: all required, phone format. API: name, phone, location required; status defaults to 'available'.
Result	Listed immediately with search and an availability filter.
Database	INSERT into `ambulances`.
API	`POST /api/ambulance/register` → 201; `GET /api/ambulance`.
Limitations	**Partially Implemented**: no endpoint to change availability or delete; the admin screen is read-only.

11.10 Admin dashboard

Purpose	Overview of all platform data.
Who / access	Administrators · `/admin/dashboard`.
Data	Totals and summaries calculated in the browser from one request.
API	`GET /api/admin/all` (admin token). Users are returned without password hashes.
Common errors	401/403 → signed out or refused; network error → "couldn't be loaded" with Try again.

11.11 Admin management & delete

Purpose	Review, search, filter and delete records.
Who / access	Administrators · `/admin/manage/:resource` (8 screens).
API	`GET /api/manage/<type>` and `DELETE /api/manage/<type>/:id` for users, hospitals, volunteers, blood-donors, food-donations, events, emergencies. Ambulances use the public `GET /api/ambulance`.
Database	DELETE … WHERE id = ?. Deleting a user sets `emergencies.user_id` to NULL (foreign key ON DELETE SET NULL).
Common errors	404 "… not found" (shown as already deleted); 400 "Invalid id."; 401/403.
Limitations	No editing, no bulk actions, no export. Search, filters and paging run in the browser over the full table.

PART C

Technical Documentation

Architecture, technology, database, authentication and the complete API reference. For developers and technical reviewers.

12. System Architecture

13. Technology Stack

14. Frontend Architecture

15. Backend Architecture

16. Database Architecture

17. Database Tables

18. Authentication & Authorization

19. API Documentation

20. HTTP Status Codes

12. System Architecture

LifeLink Pro is a classic three-tier web application: a React single-page application (SPA) in the browser, a stateless Node.js/Express REST API, and a MySQL database. The frontend and backend are separate npm projects that are built and deployed independently. They communicate only through JSON over HTTP(S).

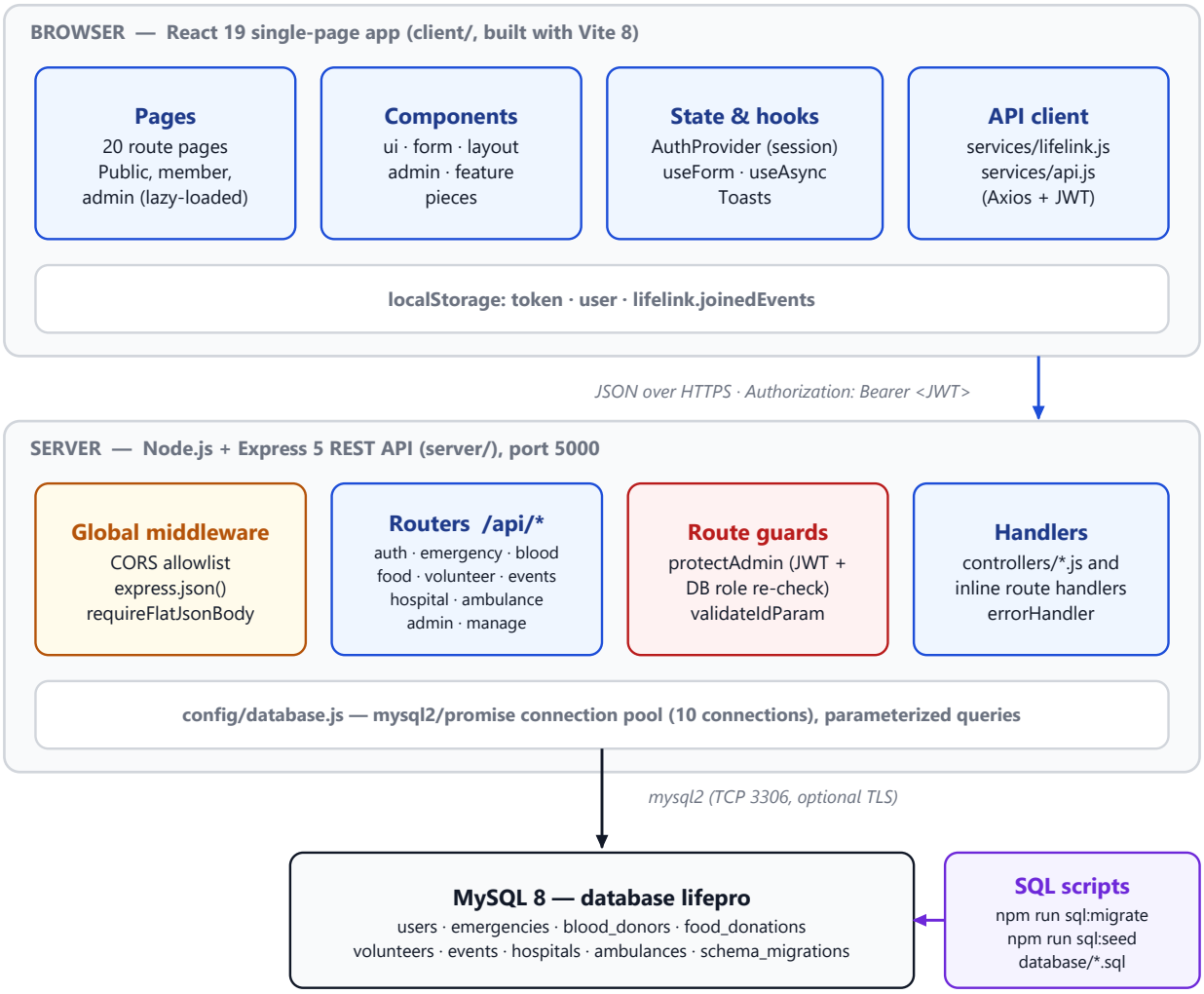


Figure 20 — System architecture (as implemented)

Property	Implementation
Communication	REST-style JSON API under `/api`; the browser calls it with Axios. Cross-origin calls are limited to the origins in `CLIENT_URL`.
State	The API keeps no sessions. Identity travels in a signed JWT; the browser keeps it in `localStorage`.
Data access	mysql2/promise connection pool (10 connections), `pool.execute` with `?` placeholders everywhere. No ORM.
Schema management	Versioned SQL files in `database/migrations`, applied by `npm run sql:migrate` and tracked in `schema_migrations`.
External services	None. No e-mail, SMS, maps, payment, analytics or cloud SDKs.
File uploads	None. `server/uploads/` exists but is empty and unused.

13. Technology Stack

Versions are the ones installed from the lockfiles (`package-lock.json`) on 26 October 2026. Ranges in `package.json` allow compatible updates.

Layer	Technology	Version	Purpose
Frontend	React + React DOM	19.2.8	User interface components and pages
Routing	React Router (react-router-dom)	7.18.2	Client-side routes, route guards
HTTP client	Axios	1.20.0	API calls; adds the JWT; handles 401
Icons	react-icons (Lucide set)	5.7.0	Interface icons
Build tool	Vite + @vitejs/plugin-react	8.1.5 / 6.0.4	Dev server and production build
Styling	CSS Modules + CSS variables	—	Component styles; design tokens in `styles/tokens.css`
Linting	oxlint	1.76.0	`npm run lint`
Runtime	Node.js	22.19.0 used	Server runtime and tooling (Node 20.19+ or 22.12+ required by Vite 8)
Package manager	npm	10.9.3 used	Dependencies and scripts
Backend framework	Express	5.2.1	REST API, routing, JSON parsing
DB driver	mysql2 (promise API)	3.24.5	Connection pool, parameterized queries
Database	MySQL Community Server	8.0.43 tested	Persistent data (InnoDB, utf8mb4)
Authentication	jsonwebtoken	9.0.3	Sign and verify JWTs (7-day expiry)
Password security	bcrypt	6.0.0	Password hashing (cost factor 10)
CORS	cors	2.8.6	Restricts browser origins
Configuration	dotenv	17.4.2	Loads `server/.env`
Dev tooling	nodemon	3.1.14	Restarts the API on changes (`npm run dev`)

Table 4 — Technology stack and versions

> **Not used**
>
> No UI framework (Bootstrap, Tailwind, Material), no chart library, no state-management library (Redux etc.), no ORM, no TypeScript, no Docker files and no CI configuration are present.

14. Frontend Architecture

The frontend (`client/`) is a Vite-built React SPA. `index.html` loads `src/main.jsx`, which wraps the app in the router and two context providers.

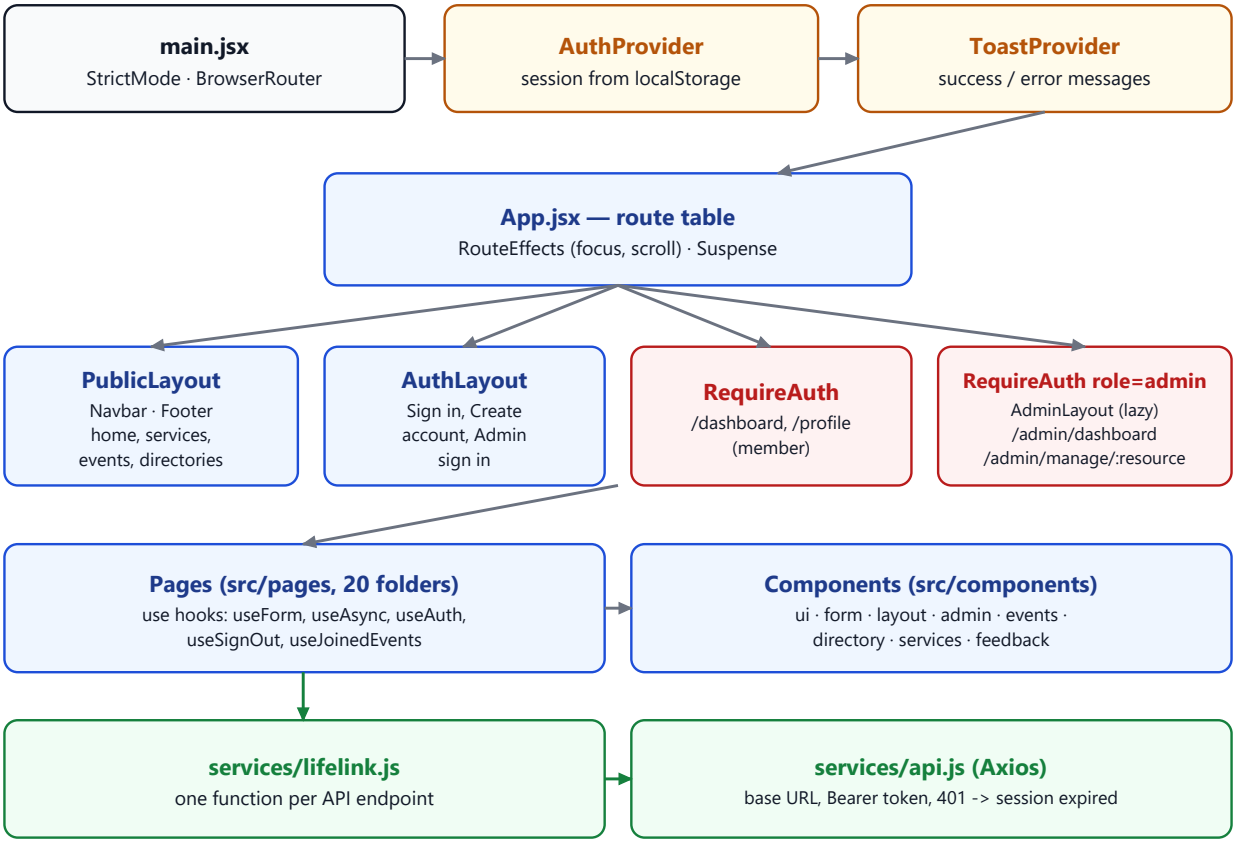


Figure 21 — Frontend architecture

14.1 Folder structure

Folder	Contents
`src/pages/`	20 page folders, one per route (Home, Login, Register, AdminLogin, Dashboard, Profile, Contact, NotFound, Emergency, BloodDonation, FoodDonation, Volunteer, Events, EventCreate, Hospitals, HospitalRegister, Ambulance, AmbulanceRegister, AdminDashboard, AdminManage).
`src/components/ui/`	Design-system pieces: Button, IconButton, Card, Badge, Alert, Spinner, Skeleton, EmptyState, ErrorState, PageHeader, SectionHeader, ConfirmDialog, Drawer, ToastProvider, SearchInput, SegmentedControl, DataTable, Toolbar.
`src/components/form/`	TextField, PasswordField, TextAreaField, ChoiceGroup, FieldShell, FormLayout.
`src/components/layout/`	Navbar, NavMenu, UserMenu, Footer, Logo, PublicLayout, AuthLayout, FormPage, SplitLayout, CardGrid, RequireAuth, RouteEffects, PageLoader.
`src/components/admin/`	AdminLayout, AdminNav, StatCard, table cells and `adminResources.jsx` (one configuration object per management screen).
`src/components/...`	Feature pieces: events, directory, services, emergency, feedback, content, auth.
`src/context/`	AuthProvider (session state) and toast context.

Folder	Contents
src/hooks/	useAuth, useForm, useAsync, useSignOut, useJoinedEvents, useDismiss, useGoBack, useDocumentTitle, useToast.
src/services/	api.js (Axios instance) and lifelink.js (one function per endpoint).
src/config/services.js	Service list, navigation groups, emergency numbers (112, 108).
src/utils/	Formatting, validation rules, status labels, error-message mapping.
src/styles/	tokens.css (design tokens) and global.css (reset, utilities).

14.2 Routes

Path	Page	Access
/	Home	Public
/contact	Help & contact (FAQ)	Public
/emergency	Emergency SOS form	Public
/blood, /food, /volunteer	Service forms	Public
/events, /events/new	Event list, create event	Public
/hospitals, /hospitals/register	Hospital directory, add hospital	Public
/ambulance, /ambulance/register	Ambulance directory, register ambulance	Public
/login, /register, /admin-login	Sign in, create account, administrator sign in	Public (signed-in users redirected)
/dashboard, /profile	Member dashboard, profile	Signed in
/admin → /admin/dashboard	Admin dashboard	Admin
/admin/manage → /admin/manage/emergencies	Management screens: :resource = emergencies, blood-donors, food-donations, volunteers, events, hospitals, ambulances, users	Admin
*	Page not found	Public

Table 5 — Frontend routes (client/src/App.jsx)

14.3 API communication

- services/api.js creates an Axios instance with base URL VITE_API_URL (default http://localhost:5000/api) and adds Authorization: Bearer <token> when a token is stored.
- A response interceptor treats any 401 on a request that carried a token as an expired session: it raises a lifelink:session-expired event; AuthProvider signs out and the sign-in page explains why.
- services/lifelink.js exposes one function per endpoint (authApi, emergencyApi, bloodApi, foodApi, volunteerApi, eventsApi, hospitalApi, ambulanceApi, adminApi, manageApi). Pages never build URLs.
- utils/errors.js turns errors into safe messages: network failure → "We couldn't reach the LifeLink Pro server…", 5xx → a generic message, 4xx → the API's own message.

14.4 State management, forms and styling

- **State:** React state and context only. AuthProvider holds the signed-in user (synchronised across browser tabs through the storage event); ToastProvider shows notifications; `useAsync` handles loading/error/data for each list.
- **Forms:** `useForm` validates with rules from `utils/validation.js` (required, email, phone, minLength, matches, wholeNumberBetween, notInPast). Errors appear after a field is left or on submit; focus moves to the first invalid field.
- **Code splitting:** the admin area (AdminLayout, AdminDashboard, AdminManage) is lazy-loaded, so public visitors never download it.
- **Styling:** CSS Modules per component; colours, spacing (4 px scale), type sizes, radii, shadows and z-index layers are CSS variables in `tokens.css`. Layouts are mobile-first and were checked from 320 to 1920 pixels wide.
- **Accessibility:** one `<h1>` and a document title per page, skip link, focus moved to the main content on navigation, labelled fields with linked hints/errors, native `<dialog>` for confirmations and the mobile menu (Escape closes).

15. Backend Architecture

The backend (`server/`) is a CommonJS Express 5 application started from `server.js`. On start it validates configuration, creates the MySQL pool, registers middleware and routers, and listens on `PORT` (default 5000).

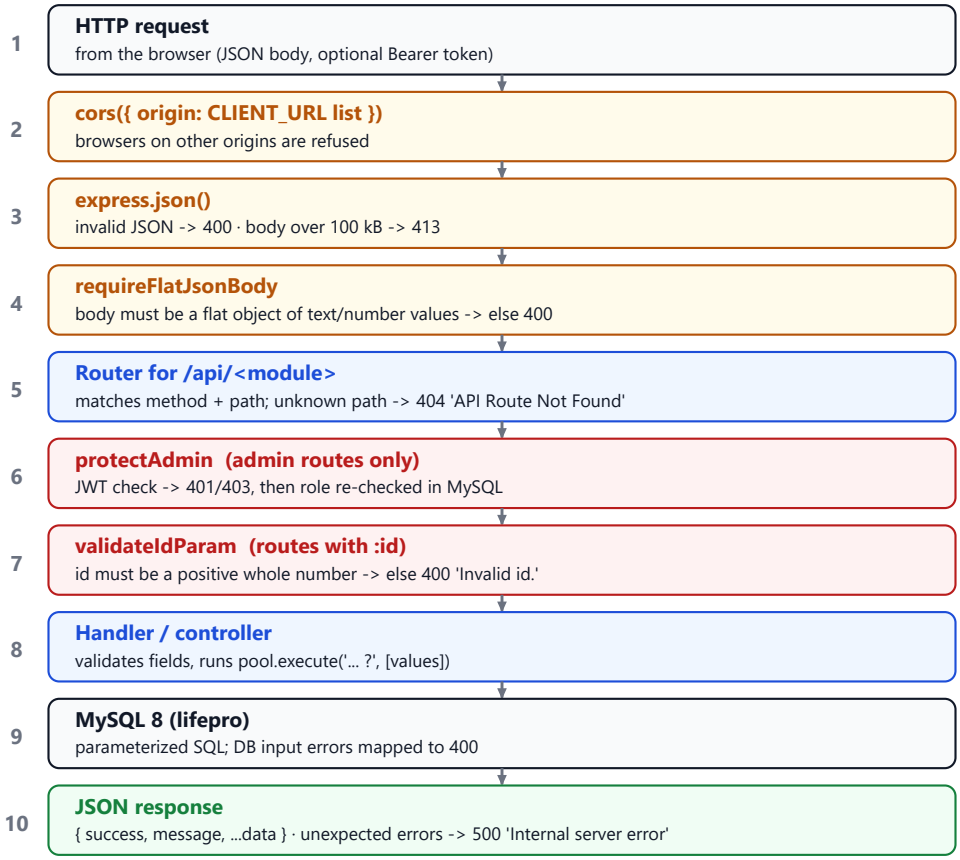


Figure 22 — Backend request flow

15.1 Files and responsibilities

File	Responsibility
`server.js`	Startup checks (required env vars; in production `CLIENT_URL` and a 32+ character `JWT_SECRET`), disables `X-Powered-By`, CORS allowlist, JSON parsing, routers, 404 handler, error handler.
`config/connectionOptions.js`	Builds MySQL connection options from `.env`; optional TLS with `DB_SSL` / `DB_SSL_CA`. Shared by the API and the SQL scripts.
`config/database.js`	mysql2/promise pool (10 connections; `DATE` columns returned as `YYYY-MM-DD` strings).
`middleware/authMiddleware.js`	`protectAdmin`: verifies the JWT, requires role admin, then re-checks the role in the database.
`middleware/validation.js`	`requireFlatJsonBody` (bodies must be flat JSON objects) and `validateIdParam` (`:id` must be a positive integer).
`middleware/errorHandler.js`	`sendServerError` maps MySQL input errors to 400 and everything else to a generic 500; `errorHandler` handles invalid JSON (400) and oversized bodies (413).
`controllers/`	authController (register, login), bloodController, emergencyController, adminController (stats, all data), manageController (lists and deletes).
`routes/`	One router per prefix: auth, emergency, blood, food, volunteer, events, hospital, ambulance, admin, manage. Food, volunteer, events, hospital and ambulance keep their handlers inline in the route file.
`scripts/migrate.js`	Applies pending SQL migrations (section 24).
`scripts/seed.js`	Loads demo data; refuses in production (section 24).
`tests/api.test.mjs`	78 end-to-end checks of every endpoint.
`tests/security.test.mjs`	71 security checks (password exposure, auth, validation, CORS, config).

15.2 Error handling

Every handler wraps its work in `try/catch`, logs the full error to the server console (`console.error`) and calls `sendServerError`. Clients only ever receive a short message; SQL text and stack traces never leave the server.

MySQL error code	HTTP status	Message sent to the client
`ER_DATA_TOO_LONG`	400	One or more fields are too long.
`ER_TRUNCATED_WRONG_VALUE`, `…_FOR_FIELD`, `WARN_DATA_TRUNCATED`	400	One or more fields have an invalid value.
`ER_WARN_DATA_OUT_OF_RANGE`	400	One or more fields are out of range.
`ER_BAD_NULL_ERROR`	400	A required field is missing.
`ER_DUP_ENTRY`	400	This record already exists.
Anything else	500	Internal server error

16. Database Architecture

All data lives in one MySQL 8 database, named by `DB_NAME` (`lifepro` by convention). The schema is defined in `database/migrations/001_initial_schema.sql`.

Convention	Value
Engine / charset	InnoDB, `utf8mb4` with `utf8mb4_unicode_ci` collation
Primary keys	`id BIGINT UNSIGNED AUTO_INCREMENT` on every business table
Timestamps	`created_at` and `updated_at` (`TIMESTAMP`, set and updated automatically by MySQL)
Naming	snake_case columns (`blood_group`); the API accepts camelCase request fields (`bloodGroup`) and maps them
Required fields	`NOT NULL` exactly where the API validates a field; other columns are nullable
Foreign keys	One: `emergencies.user_id → users.id` (ON DELETE SET NULL, ON UPDATE CASCADE)
Unique keys	`users.email`, `blood_donors.phone`
Bookkeeping	`schema_migrations` records applied migration files

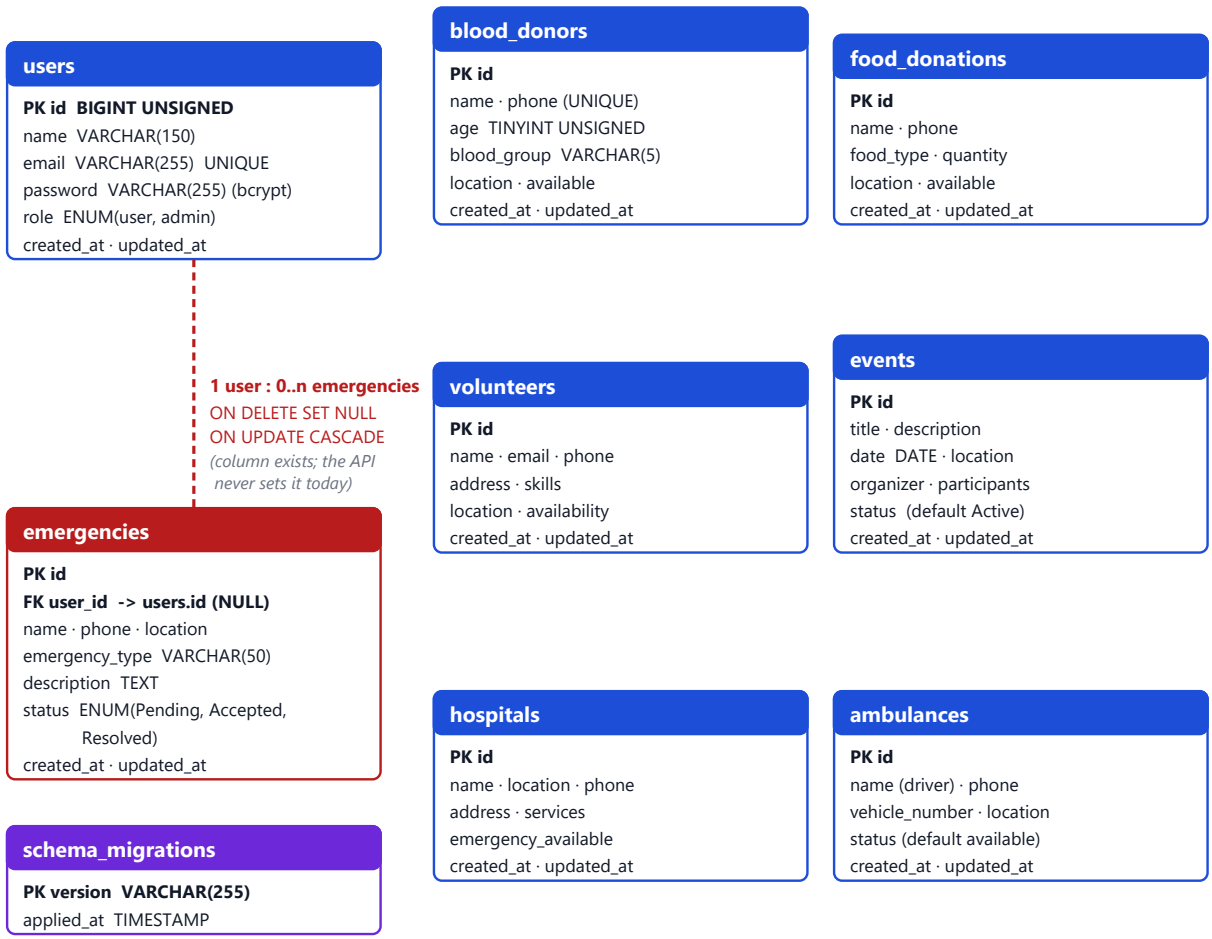


Figure 23 — Database entity-relationship diagram

> **Relationships**
>
> Only users and emergencies are related. All other tables are independent: for example a blood donor or volunteer record is not linked to a user account, and events do not store who joined.

17. Database Tables

17.1 users

Accounts for sign-in. Written by POST /api/auth/register; read by login, protectAdmin and the admin screens.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
name	VARCHAR(150)	No	—	Full name
email	VARCHAR(255)	No	—	Login email (unique)
password	VARCHAR(255)	No	—	bcrypt hash, never plain text; never returned by the API
role	ENUM('user','admin')	No	'user'	Account type
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

Indexes: PRIMARY (id), UNIQUE uq_users_email (email), idx_users_created_at.

17.2 emergencies

SOS requests. Written by POST /api/emergency; managed under /api/manage/emergencies.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
user_id	BIGINT UNSIGNED	Yes	NULL	FK → users.id; not set by the current API
name	VARCHAR(150)	No	—	Requester name
emergency_type	VARCHAR(50)	Yes	NULL	Medical, Accident, Fire, Other
location	VARCHAR(255)	No	—	Where help is needed
phone	VARCHAR(30)	No	—	Call-back number
description	TEXT	Yes	NULL	What happened
status	ENUM('Pending','Accepted','Resolved')	No	'Pending'	No endpoint changes it
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

Indexes: PRIMARY, idx_emergencies_user_id, idx_emergencies_phone, idx_emergencies_status, idx_emergencies_created_at. Foreign key fk_emergencies_user ON DELETE SET NULL ON UPDATE CASCADE.

17.3 blood_donors

Blood donor registry. Written by POST /api/blood/register.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
name	VARCHAR(150)	No	—	Donor name
phone	VARCHAR(30)	No	—	Unique per donor
age	TINYINT UNSIGNED	Yes	NULL	18–65 enforced by the form
blood_group	VARCHAR(5)	No	—	A+, A-, B+, B-, AB+, AB-, O+, O-
location	VARCHAR(255)	No	—	City or area
available	BOOLEAN	No	TRUE	Not changed by any endpoint
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

Indexes: PRIMARY, UNIQUE uq_blood_donors_phone, idx_blood_donors_blood_group, idx_blood_donors_created_at.

17.4 food_donations

Food offered for pickup. Written by POST /api/food/donate.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
name	VARCHAR(150)	No	—	Donor name (API field donorName)
phone	VARCHAR(30)	No	—	Contact number
food_type	VARCHAR(100)	No	—	API field foodType
quantity	VARCHAR(100)	No	—	Free text, e.g. “20 plates”
location	VARCHAR(255)	No	—	Pickup location
available	BOOLEAN	No	TRUE	Shown as status; never changed
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

Indexes: PRIMARY, idx_food_donations_phone, idx_food_donations_created_at.

17.5 volunteers

Volunteer register. Written by POST /api/volunteer/register.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
name	VARCHAR(150)	No	—	Volunteer name
email	VARCHAR(255)	Yes	NULL	Optional
phone	VARCHAR(30)	Yes	NULL	Required by the form, optional in the API
address	VARCHAR(255)	Yes	NULL	Accepted by the API; not on the form
skills	VARCHAR(255)	Yes	NULL	Free text
location	VARCHAR(255)	Yes	NULL	Where they would like to help
availability	VARCHAR(50)	Yes	NULL	Full Time, Part Time, Weekends
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

Indexes: PRIMARY, idx_volunteers_created_at.

17.6 events

Community events. Written by POST /api/events/create; participants incremented by PUT /api/events/join/:id.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
title	VARCHAR(200)	No	—	API field title or eventName
description	TEXT	Yes	NULL	Optional
date	DATE	Yes	NULL	Returned as YYYY-MM-DD
location	VARCHAR(255)	Yes	NULL	Venue
organizer	VARCHAR(150)	Yes	NULL	Optional
participants	INT UNSIGNED	No	0	Join counter
status	VARCHAR(20)	No	'Active'	Not used by the application
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

Indexes: PRIMARY, idx_events_date, idx_events_created_at.

17.7 hospitals

Hospital directory. Written by POST /api/hospital/register.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
name	VARCHAR(200)	No	—	API field name or hospitalName
location	VARCHAR(255)	Yes	NULL	Accepted by the API; not on the form
phone	VARCHAR(30)	Yes	NULL	Contact number
address	VARCHAR(255)	Yes	NULL	Street address
services	TEXT	Yes	NULL	Free text
emergency_available	VARCHAR(30)	Yes	NULL	Available / Not Available
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

Indexes: PRIMARY, idx_hospitals_created_at.

17.8 ambulances

Ambulance directory. Written by POST /api/ambulance/register.

Column	Type	Null	Default	Description
id	BIGINT UNSIGNED	No	AUTO_INCREMENT	Primary key
name	VARCHAR(150)	No	—	Driver name (API name or driverName)
phone	VARCHAR(30)	No	—	Driver phone (public)
vehicle_number	VARCHAR(30)	Yes	NULL	API field vehicleNumber
location	VARCHAR(255)	No	—	Base location
status	VARCHAR(20)	No	'available'	API status or availability
created_at	TIMESTAMP	No	CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	No	CURRENT_TIMESTAMP, on update	Last change time

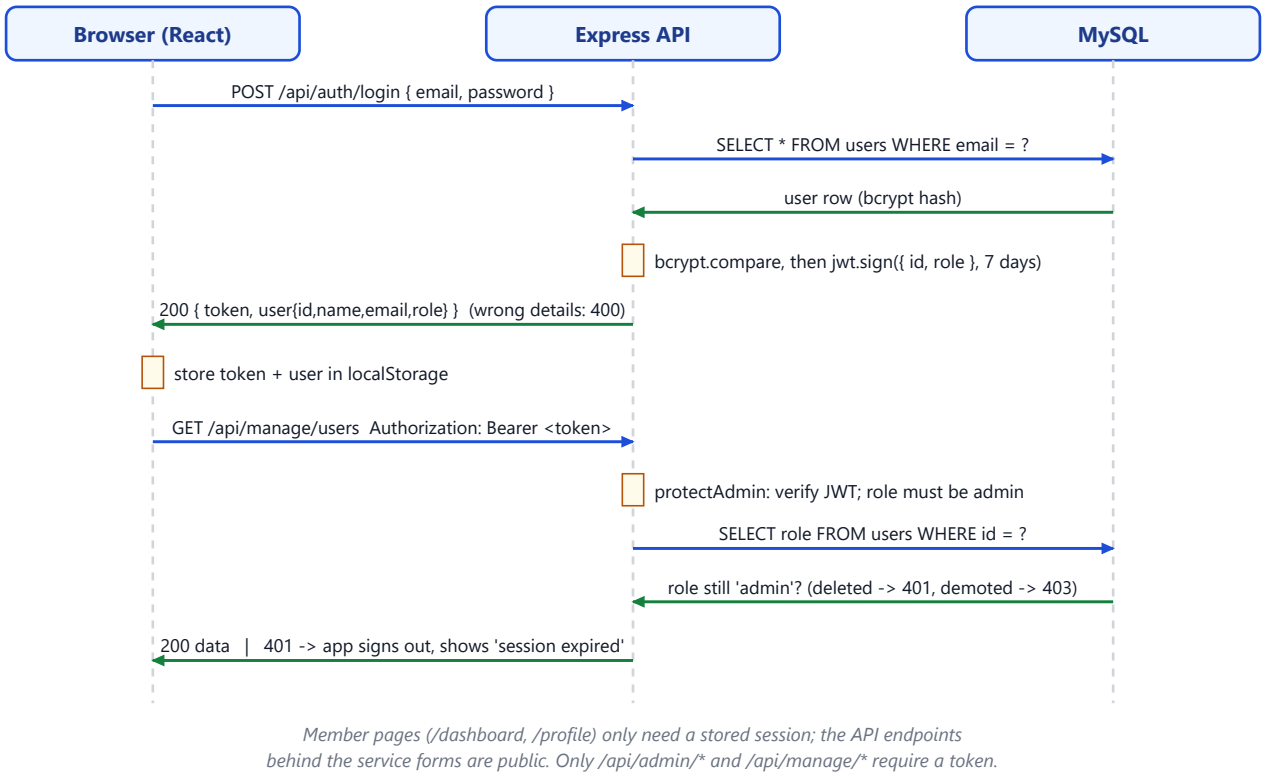
Indexes: PRIMARY, idx_ambulances_created_at.

17.9 schema_migrations

Created by `scripts/migrate.js` (not by a migration file). One row per applied migration.

Column	Type	Null	Default	Description
version	VARCHAR(255)	No	—	Primary key; migration file name
applied_at	TIMESTAMP	No	CURRENT_TIMESTAMP	When it was applied

18. Authentication & Authorization



Member pages (/dashboard, /profile) only need a stored session; the API endpoints behind the service forms are public. Only /api/admin/ and /api/manage/* require a token.*

Figure 24 — Authentication and admin authorization flow

18.1 Registration and password storage

- POST /api/auth/register requires name, email and password, checks the email format and uniqueness, and stores `bcrypt.hash(password, 10)`. The plain password is never stored or logged.
- Self-registration always creates role `user`. Administrators are created outside the app (section 27).
- The 8-character minimum is enforced by the sign-up form only; the API accepts any non-empty password (**Partially Implemented**).

18.2 Login and tokens

- POST /api/auth/login compares the password with bcrypt and returns a JWT signed with `JWT_SECRET` containing `{ id, role }`, valid for 7 days, plus `{ id, name, email, role }`.
- Unknown email and wrong password give the same 400 response, so the API does not reveal which accounts exist.
- The browser stores the token and user in `localStorage` and sends the token as `Authorization: Bearer <token>`. There is no refresh token; after 7 days the user signs in again.
- Sign-out removes the token in the browser. Tokens cannot be revoked server-side before they expire, except that admin access stops immediately when the account is deleted or demoted (section 18.3).

18.3 Authorization

Layer	Rule	Where
Admin API	Every `/api/admin/*` and `/api/manage/*` route runs `protectAdmin`: missing/invalid/expired token → 401; role not admin → 403; then `SELECT role FROM users WHERE id = ?`: account gone → 401, no longer admin → 403.	`middleware/authMiddleware.js`
Public API	All other endpoints (service forms, directories, events, auth) need no token.	`routes/*.js`
Member pages	`RequireAuth` redirects signed-out users to /login (remembering the page).	`components/layout/RequireAuth.jsx`
Admin pages	`RequireAuth role="admin"` sends signed-out users to /admin-login and members to /dashboard with a notice.	`App.jsx`
Admin sign-in	Only starts a session if the returned role is admin.	`pages/AdminLogin`

> **Browser checks are convenience, not security**
>
> Hiding pages in the browser can always be bypassed. The real protection is protectAdmin on the server, which is applied to every admin endpoint and covered by the security test suite.

19. API Documentation

Base URL	`http://localhost:5000/api` in development; in production the HTTPS address of your API plus `/api`.
Format	JSON request and response bodies. Every response contains `success: true|false`.
Errors	`{ "success": false, "message": "..." }` — see section 20.
Bodies	Must be flat JSON objects (text, number, boolean or null values). Nested objects or arrays → 400. Maximum size 100 kB → 413.
Authentication	Admin endpoints need `Authorization: Bearer <JWT>` from `/api/auth/login` for an admin account.
Field names	Requests use the camelCase names the forms send; responses return database rows with snake_case columns (the manage endpoints add camelCase aliases).

19.1 Endpoint overview

Method	Path	Purpose	Auth
GET	/	API health / welcome message	Public
POST	/api/auth/register	Create a member account	Public
POST	/api/auth/login	Sign in, receive a JWT	Public
POST	/api/emergency	Create an emergency request	Public
GET	/api/emergency	List emergency requests	Public
POST	/api/blood/register	Register a blood donor	Public
GET	/api/blood	List blood donors	Public
POST	/api/food/donate	Create a food donation	Public
GET	/api/food	List food donations	Public
POST	/api/volunteer/register	Register a volunteer	Public
GET	/api/volunteer	List volunteers	Public
POST	/api/events/create	Create an event	Public
GET	/api/events	List events	Public
PUT	/api/events/join/:id	Join an event (participants + 1)	Public
POST	/api/hospital/register	Register a hospital	Public
GET	/api/hospital	List hospitals	Public
POST	/api/ambulance/register	Register an ambulance	Public
GET	/api/ambulance	List ambulances	Public
GET	/api/admin/stats	Record counts (not used by the UI)	Admin
GET	/api/admin/all	All records for the admin dashboard	Admin
GET	/api/manage/dashboard	Record counts (not used by the UI)	Admin
GET	/api/manage/users	List users	Admin
DELETE	/api/manage/users/:id	Delete one user	Admin
GET	/api/manage/hospitals	List hospitals	Admin

Method	Path	Purpose	Auth
DELETE	/api/manage/hospitals/:id	Delete one hospital	Admin
GET	/api/manage/volunteers	List volunteers	Admin
DELETE	/api/manage/volunteers/:id	Delete one volunteer	Admin
GET	/api/manage/blood-donors	List blood donors	Admin
DELETE	/api/manage/blood-donors/:id	Delete one blood donor	Admin
GET	/api/manage/food-donations	List food donations	Admin
DELETE	/api/manage/food-donations/:id	Delete one food donation	Admin
GET	/api/manage/events	List events	Admin
DELETE	/api/manage/events/:id	Delete one event	Admin
GET	/api/manage/emergencies	List emergencies	Admin
DELETE	/api/manage/emergencies/:id	Delete one emergencie	Admin

Table 6 — All 35 API endpoints (verified in server/server.js and server/routes)

Any other path returns 404 `{ "success": false, "message": "API Route Not Found" }`.

19.2 Health

GET /

Purpose	Confirms the API is running.
Authentication	None

Successful response

```
{ "success": true, "message": "<rocket emoji> Welcome to LifeLink Pro API" }
```

19.3 Authentication

POST /api/auth/register

Purpose	Create a member account (role is always "user").
Authentication	None
Errors	400 “Name, email and password are required.” · 400 “Please provide a valid email address.” · 400 “User already exists” · 500

Request body

```
{ "name": "Asha Rao", "email": "asha@example.com", "password": "********" }
```

Successful response

```
201
{ "success": true, "message": "Registration successful",
  "user": { "id": 3, "name": "Asha Rao", "email": "asha@example.com", "role": "user" } }
```

POST /api/auth/login

Purpose	Authenticate and receive a JWT.
Authentication	None
Errors	400 “Email and password are required.” · 400 “Invalid email or password” · 500

Request body

```
{ "email": "asha@example.com", "password": "********" }
```

Successful response

```
200
{ "success": true, "message": "Login successful", "token": "<JWT>",
  "user": { "id": 3, "name": "Asha Rao", "email": "asha@example.com", "role": "user" } }
```

19.4 Emergency requests

POST /api/emergency

Purpose	Create an SOS request. Status starts as Pending.
Authentication	None
Notes	name, phone, location required; emergencyType and description optional.
Errors	400 “Name, phone and location are required.” · 400 “Emergency request already exists.” (same name + phone + location) · 400 database input errors · 500

Request body

```
{ "name": "Ravi", "phone": "98765 43210", "location": "MG Road, near bus stand",
  "emergencyType": "Medical", "description": "Elderly man fainted" }
```

Successful response

```
201
{ "success": true, "message": "Emergency request created successfully.",
  "emergency": { "id": 15, "user_id": null, "name": "Ravi", "emergency_type": "Medical",
                 "location": "...", "phone": "...", "description": "...",
                 "status": "Pending", "created_at": "...", "updated_at": "..." } }
```

GET /api/emergency

Purpose	List all emergency requests, newest first.
Authentication	None
Notes	Public and unused by the web app. Returns personal data (see section 34).

Successful response

```
{ "success": true, "emergencies": [ { ...emergency row... } ] }
```

19.5 Blood donors

POST /api/blood/register

Purpose	Register a blood donor.
Authentication	None
Notes	age optional in the API (form requires 18–65).
Errors	400 "Name, phone, blood group and location are required." · 400 "You are already registered as a blood donor" · 400 database input errors (e.g. age out of range) · 500

Request body

```
{ "name": "Meena", "phone": "9123456780", "age": "30", "bloodGroup": "O-", "location": "Pune" }
```

Successful response

```
201
{ "success": true, "message": "Blood donor registered successfully",
  "donor": { "id": 9, "name": "Meena", "phone": "...", "age": 30, "blood_group": "O-",
             "location": "Pune", "available": 1, "created_at": "...", "updated_at": "..." } }
```

GET /api/blood

Purpose	List blood donors, newest first.
Authentication	None
Notes	Public; not called by the web app (removed so donor phone numbers are not shown publicly).

Successful response

```
{ "success": true, "donors": [ { ...blood_donors row... } ] }
```

19.6 Food donations

POST /api/food/donate

Purpose	Create a food donation.
Authentication	None
Notes	All five fields required.
Errors	400 "Donor name, phone, food type, quantity and location are required." · 400 "This food donation already exists." · 500

Request body

```
{ "donorName": "Hotel Green", "phone": "9123456781", "foodType": "Cooked meals",
  "quantity": "20 plates", "location": "12 Station Road" }
```

Successful response

```
201
{ "success": true, "message": "Food donation added successfully.",
  "donation": { "id": 4, "name": "Hotel Green", "food_type": "Cooked meals", ... } }
```

GET /api/food

Purpose	List food donations, newest first.
Authentication	None
Notes	Public; not used by the web app.

Successful response

```
{ "success": true, "donations": [ { ...food_donations row... } ] }
```

19.7 Volunteers

POST /api/volunteer/register

Purpose	Register a volunteer.
Authentication	None
Notes	Only name is required by the API; address also accepted.
Errors	400 “Volunteer name is required.” · 500

Request body

```
{ "name": "Kiran", "phone": "9123456782", "email": "kiran@example.com",
  "skills": "First aid", "location": "Pune", "availability": "Weekends" }
```

Successful response

```
201
{ "success": true, "message": "Volunteer registered successfully", "volunteer": { ... } }
```

GET /api/volunteer

Purpose	List volunteers, newest first.
Authentication	None
Notes	Public; not used by the web app.

Successful response

```
{ "success": true, "volunteers": [ ... ] }
```

19.8 Events

POST /api/events/create

Purpose	Create an event (participants start at 0).
Authentication	None
Notes	Accepts title or eventName.
Errors	400 “Event title is required.” · 400 invalid date · 500

Request body

```
{ "eventName": "Blood donation camp", "description": "...", "date": "2026-12-20",
  "location": "City Hall", "organizer": "Red Cross volunteers" }
```

Successful response

```
201
{ "success": true, "message": "Event created successfully",
  "event": { "id": 17, "title": "Blood donation camp", "date": "2026-12-20",
             "participants": 0, "status": "Active", ... } }
```

GET /api/events

Purpose	List events (newest created first).
Authentication	None

Successful response

```
{ "success": true, "events": [ ... ] }
```

PUT /api/events/join/:id

Purpose	Increase an event's participant count by one.
Authentication	None
Notes	No body. Anonymous: does not record who joined and can be repeated.
Errors	400 “Invalid id.” · 404 “Event not found” · 500

Successful response

```
{ "success": true, "message": "Joined event successfully", "event": { ...updated row... } }
```

19.9 Hospitals

POST /api/hospital/register

Purpose	Register a hospital.
Authentication	None
Notes	Accepts name or hospitalName; location also accepted.
Errors	400 “Hospital name is required.” · 500

Request body

```
{ "hospitalName": "City Care Hospital", "phone": "020 1234 5678",
  "address": "2 Hill Road, Pune", "services": "Emergency, ICU", "emergencyAvailable": "Available" }
```

Successful response

```
201
{ "success": true, "message": "Hospital registered successfully", "hospital": { ... } }
```

GET /api/hospital

Purpose	List hospitals, newest first.
Authentication	None
Notes	Used by the public hospital directory.

Successful response

```
{ "success": true, "hospitals": [ ... ] }
```

19.10 Ambulances

POST /api/ambulance/register

Purpose	Register an ambulance.
Authentication	None
Notes	Accepts name or driverName, status or availability (default "available").
Errors	400 “Name, phone and location are required.” · 500

Request body

```
{ "driverName": "Suresh", "phone": "9123456784", "vehicleNumber": "MH-12-AB-1234",
  "location": "Pune North", "availability": "Available" }
```

Successful response

```
201
{ "success": true, "message": "Ambulance registered successfully", "ambulance": { ... } }
```

GET /api/ambulance

Purpose	List ambulances, newest first.
Authentication	None
Notes	Used by the public directory and the admin ambulances screen.

Successful response

```
{ "success": true, "ambulances": [ ... ] }
```

19.11 Admin

GET /api/admin/stats

Purpose	Counts of users, blood donors, food donations, volunteers, events, hospitals and ambulances.
Authentication	Admin JWT
Notes	Not used by the current web app.
Errors	401 "No token provided" / "Invalid token" / "Token expired or invalid" · 403 "Admin access required" · 500

Successful response

```
{ "success": true, "stats": { "users": 2, "bloodDonors": 2, "foodDonations": 1,
  "volunteers": 1, "events": 1, "hospitals": 1, "ambulances": 1 } }
```

GET /api/admin/all

Purpose	Every record of every table for the admin dashboard.
Authentication	Admin JWT
Notes	Users are returned as id, name, email, role, created_at, updated_at (no password).
Errors	401 "No token provided" / "Invalid token" / "Token expired or invalid" · 403 "Admin access required" · 500

Successful response

```
{ "success": true, "data": { "users": [...], "bloodDonors": [...], "foodDonations": [...],
  "volunteers": [...], "events": [...], "hospitals": [...], "ambulances": [...],
  "emergencies": [...] } }
```

19.12 Admin management

GET /api/manage/dashboard

Purpose	Counts of users, hospitals, volunteers, blood donors, food donations, events and emergencies.
Authentication	Admin JWT
Notes	Not used by the current web app.
Errors	401 "No token provided" / "Invalid token" / "Token expired or invalid" · 403 "Admin access required" · 500

Successful response

```
{ "success": true, "stats": { ... } }
```

List endpoint	Response key	Extra camelCase fields	Delete endpoint → 404 message
GET /api/manage/users	users	— (password never selected)	DELETE .../users/:id → "User not found"
GET /api/manage/hospitals	hospitals	hospitalName, emergencyAvailable	DELETE .../hospitals/:id → "Hospital not found"

List endpoint	Response key	Extra camelCase fields	Delete endpoint → 404 message
GET /api/manage/volunteers	volunteers	—	DELETE .../volunteers/:id → "Volunteer not found"
GET /api/manage/blood-donors	donors	bloodGroup	DELETE .../blood-donors/:id → "Blood donor not found"
GET /api/manage/food-donations	foodDonations	donorName, foodType	DELETE .../food-donations/:id → "Food donation not found"
GET /api/manage/events	events	eventName	DELETE .../events/:id → "Event not found"
GET /api/manage/emergencies	emergencies	emergencyType	DELETE .../emergencies/:id → "Emergency not found"

All 15 manage endpoints require an admin JWT and return the admin errors above. Lists are ordered newest first. Successful deletes return `200 { "success": true, "message": "<Type> deleted successfully" }`; an invalid id returns 400 "Invalid id.".

> **Delete has no self-protection in the API**
>
> DELETE /api/manage/users/:id will delete any user, including the administrator making the request. Only the web app hides the Delete button on your own account.

20. HTTP Status Codes

Status	When it is returned	Example message
200 OK	Successful reads, login, join event, delete	"Login successful"
201 Created	Successful register / create endpoints (all POSTs except login)	"Event created successfully"
400 Bad Request	Missing or invalid fields, duplicates, wrong login details, invalid JSON, non-flat body, invalid :id, values the database rejects	"Invalid email or password", "Invalid id."
401 Unauthorized	Admin endpoint without a valid token, expired token, or the account no longer exists	"No token provided", "Token expired or invalid"
403 Forbidden	Valid token but the account is not (or no longer) an admin	"Admin access required"
404 Not Found	Unknown path; event to join or record to delete does not exist	"API Route Not Found", "Event not found"
413 Payload Too Large	Request body over 100 kB	"Request body is too large."
500 Internal Server Error	Unexpected server or database failure (details only in the server log)	"Internal server error"

Table 7 — HTTP status codes returned by the API

Note: a failed login returns 400, not 401. In the web app, a 401 on an authenticated request ends the session.

PART D

Setup, Deployment & Operations

Installing, configuring, running, deploying, securing, monitoring, troubleshooting and backing up LifeLink Pro.

21. Installation Guide
22. MySQL Installation & Database Setup
23. Environment Configuration
24. Database Migration & Seed
25. Running the Application
26. Deployment Guide
27. Production Configuration & Administrator Accounts
28. Security
29. Monitoring & Maintenance
30. Troubleshooting
31. Backup & Recovery

21. Installation Guide

These steps set up a development copy on a new computer (Windows, macOS or Linux).

21.1 Requirements

Software	Version	Notes
Node.js	20.19+ or 22.12+	Required by Vite 8. Tested with 22.19.0. Includes npm.
npm	10+	Tested with 10.9.3.
MySQL Server	8.0+	Tested with 8.0.43 Community Server. MariaDB is not tested.
Git	any	Optional. The delivered project folder is not a Git repository and no repository URL is recorded in the project.
Web browser	current	Chrome, Edge, Firefox or Safari.

21.2 Get the source code

Copy the delivered `lifepro` folder to the computer, or clone your organisation's repository if you have created one:

```
git clone <your-repository-url> lifepro
cd lifepro
```

21.3 Install dependencies

There is no `package.json` in the project root. Install the backend and frontend separately:

```
cd server
npm install          # Express, mysql2, bcrypt, jsonwebtoken, cors, dotenv, nodemon

cd ../client
npm install          # React, React Router, Axios, react-icons, Vite, oxlint
```

> **Reproducible installs**
>
> Use `npm ci` instead of `npm install` to install exactly the versions in `package-lock.json`. If `node_modules` is ever deleted, run it again; otherwise `npm run dev` fails with "'vite' is not recognized".

Then continue with MySQL (section 22), configuration (section 23) and the database (section 24).

22. MySQL Installation & Database Setup

1. **Install MySQL Server 8.0+.** Windows: MySQL Installer ("Server only"; it creates the service `MySQL80`). macOS: `brew install mysql`. Ubuntu/Debian: `sudo apt install mysql-server`.
2. **Start MySQL.** Windows: Services → MySQL80 → Start, or `net start MySQL80` in an administrator terminal. macOS: `brew services start mysql`. Linux: `sudo systemctl start mysql`.
3. **Check it works:** `mysql -u root -p -e "SELECT VERSION();"`
4. **Create a database user (recommended).** The migration creates the database automatically if the configured user may; otherwise create it yourself (example below).
5. **Configure** `server/.env` with these credentials (section 23).
6. **Run the migration:** `cd server && npm run sql:migrate` (section 24).

Example only — replace the placeholder

```
CREATE DATABASE lifepro CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'lifepro'@'localhost' IDENTIFIED BY '<choose-a-strong-password>';
GRANT ALL PRIVILEGES ON lifepro.* TO 'lifepro'@'localhost';
```

Never create tables by hand (for example in phpMyAdmin): the migration is the single source of truth for the schema.

23. Environment Configuration

The API reads its settings from `server/.env` (copy `server/.env.example`). In production you can set the same names as real environment variables instead.

Variable	Required	Example / default	Purpose
`NODE_ENV`	No	development	`production` enables extra startup checks and blocks the demo seed
`PORT`	No	5000	API port
`CLIENT_URL`	In production	http://localhost:5173	Allowed browser origin(s) for CORS, comma-separated, no trailing slash needed
`DB_HOST`	No	localhost	MySQL host
`DB_PORT`	No	3306	MySQL port
`DB_USER`	Yes	<mysql user>	MySQL user (avoid root in production)
`DB_PASSWORD`	Usually	<mysql password>	MySQL password
`DB_NAME`	Yes	lifepro	Database name (created by the migration if allowed)
`DB_SSL`	No	false	`true` for an encrypted connection to a hosted MySQL
`DB_SSL_CA`	No	/path/to/ca.pem	Provider's CA certificate when DB_SSL=true
`JWT_SECRET`	Yes	<long random value>	Signs login tokens; at least 32 characters in production

Table 8 — Backend environment variables (server/.env.example)

server/.env — placeholders only

```
NODE_ENV=development
PORT=5000
CLIENT_URL=http://localhost:5173

DB_HOST=localhost
DB_PORT=3306
DB_USER=<mysql-user>
DB_PASSWORD=<mysql-password>
DB_NAME=lifepro
DB_SSL=false

JWT_SECRET=<long-random-value>
```

Generate a strong secret with:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

23.1 Frontend variable

Variable	File	Default	Purpose
VITE_API_URL	client/.env (copy client/.env.example)	http://localhost:5000/api	API base URL. Read at build time; it ends up in the public JavaScript, so it is not a secret.

> **Never share or commit server/.env**
>
> It contains the database password and the JWT secret. `.gitignore` excludes `.env` and `.env.*` (but keeps `.env.example`). The delivered folder currently contains a developer's local `server/.env`; remove it before passing the source code to anyone else.

24. Database Migration & Seed

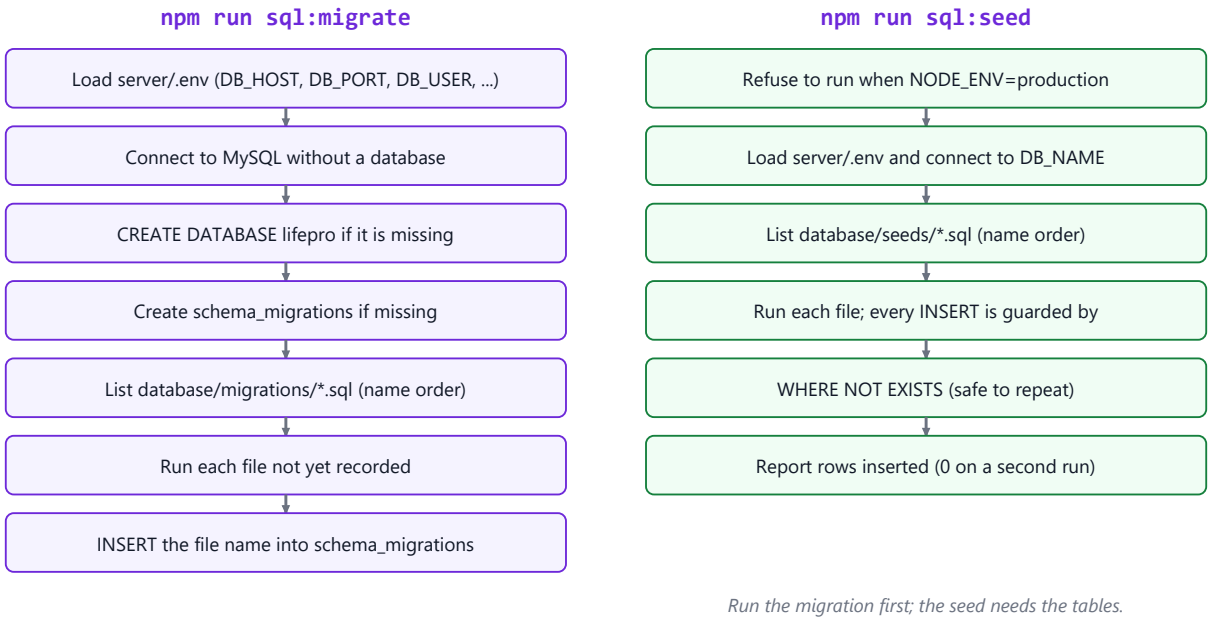


Figure 25 — Database migration and seed flow

24.1 Migration

Create or update the schema (safe to repeat)

```
cd server
npm run sql:migrate
```

- Creates the database if it does not exist (utf8mb4_unicode_ci), then the `schema_migrations` table.
- Applies every file in `database/migrations/` not yet recorded, in file-name order, and records each one. Today there is one file: `001_initial_schema.sql` (8 tables).
- Re-running prints "is up to date — no pending migrations". If the database already contains tables that the script did not create, it warns and leaves them unchanged.
- Clear hints are printed for common errors (MySQL not running, wrong credentials, missing privileges).

24.2 Seed (development and demo only)

Load demo data (safe to repeat)

```
cd server
npm run sql:seed
```

- Runs every file in `database/seeds/`. Each INSERT is guarded by `WHERE NOT EXISTS`, so a second run inserts 0 rows.
- Refuses to run when `NODE_ENV=production`.
- Loads 10 demo rows: 2 users (one administrator, one member), 2 blood donors, 1 food donation, 1 volunteer, 1 event, 1 hospital, 1 ambulance and 1 emergency request. All use "Demo …" names, `@lifelinkpro.test` emails and phone numbers 9000000001–9000000007.
- The demo login details are listed at the top of `database/seeds/001_initial_seed.sql`. They are intentionally not printed in this document; they must never exist in production.

24.3 Changing the schema later

1. Add a new file such as `database/migrations/002_add_event_participants.sql`. Never edit a migration that has already been applied.
2. Write plain SQL statements separated by semicolons (no DELIMITER blocks), compatible with MySQL 8.0.
3. Run `npm run sql:migrate` on every environment. Back up production first: MySQL commits DDL immediately, so a half-applied migration must be fixed by hand.

24.4 Resetting a local database

Deletes all local data — never run against production

```
mysql -u <user> -p -e "DROP DATABASE lifepro"
cd server
npm run sql:migrate
npm run sql:seed
```

25. Running the Application

25.1 Development (two terminals)

```
# terminal 1 — API on http://localhost:5000
cd server
npm run dev

# terminal 2 — web app on http://localhost:5173
cd client
npm run dev
```

Open http://localhost:5173. The API answers http://localhost:5000/ with its welcome message. The API must run for any page to show data.

25.2 npm scripts

Folder	Command	What it does
server	`npm start`	Start the API with Node (production)
server	`npm run dev`	Start the API with nodemon (restarts on changes)
server	`npm run sql:migrate`	Create/update the database schema
server	`npm run sql:seed`	Load demo data (not in production)
server	`npm run test:api`	78 end-to-end API checks (API running, demo seed loaded)
server	`npm run test:security`	71 security checks (same requirements)
client	`npm run dev`	Vite development server on port 5173
client	`npm run build`	Production build into client/dist
client	`npm run preview`	Serve the production build locally
client	`npm run lint`	oxlint

> **Port 5173 must be free**
>
> If another Vite server already uses 5173, Vite starts on 5174, which the API's CORS setting refuses. Stop the other server, or add the new address to `CLIENT_URL`.

26. Deployment Guide

> **Deployment status: Not verified**
>
> The project contains no hosting configuration, Dockerfile or CI pipeline, and it has not been deployed to a production server as part of this work. The guidance below follows the project's design and is provider-neutral.

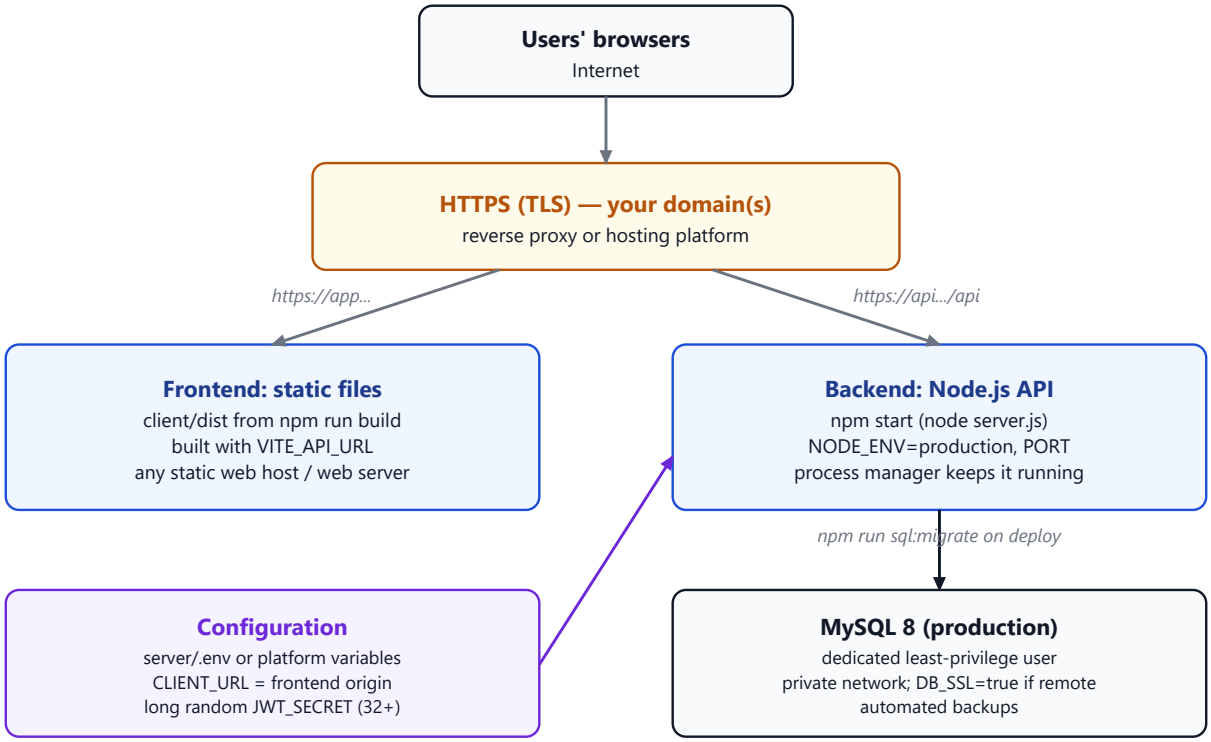


Figure 26 — Recommended production deployment architecture

26.1 Frontend deployment

1. Build with the public API address: `cd client && VITE_API_URL=https://api.example.com/api npm run build` (on Windows PowerShell set `$env:VITE_API_URL` first).
2. Upload the contents of `client/dist/` to any static web host or web server.
3. **Configure an SPA fallback:** every unknown path must serve `index.html` (the app uses browser routing). Without it, refreshing /events or /admin/dashboard returns the server's 404 page.
4. Serve it over HTTPS.

26.2 Backend deployment

1. Install Node.js (LTS, 20.19+ or 22.12+) on the server and copy `server/` (without `node_modules` and without a development `.env`).
2. Install production dependencies: `npm ci --omit=dev`.
3. Provide the environment variables (section 27).
4. Run `npm run sql:migrate` against the production database (never the seed).
5. Start with `npm start` under a process manager (for example pm2 or a systemd service) so it restarts after crashes and reboots.
6. Put it behind a reverse proxy or platform router that terminates HTTPS and forwards to `PORT`.

26.3 MySQL, domains, HTTPS and CORS

- **MySQL:** managed or self-hosted MySQL 8; a dedicated user with privileges only on the app database; not reachable from the internet; `DB_SSL=true` (and `DB_SSL_CA`) when the connection crosses a network; automated backups (section 31).
- **Domains:** typically `app.example.com` for the web app and `api.example.com` for the API. Serving both from one domain with `/api` proxied to the API also works.
- **HTTPS:** required for both, because passwords and tokens travel in requests.
- **CORS:** set `CLIENT_URL` to the exact web-app origin, e.g. `https://app.example.com` (several origins comma-separated). The API refuses to start in production without it.

26.4 Build and release commands

Example (replace example.com with your domains)

```
# frontend
cd client
npm ci
VITE_API_URL=https://api.example.com/api npm run build   # -> client/dist

# backend
cd server
npm ci --omit=dev
npm run sql:migrate
NODE_ENV=production npm start
```

27. Production Configuration & Administrator Accounts

27.1 Production settings

Setting	Production value	Enforced by the code?
NODE_ENV	production	Enables the checks below
CLIENT_URL	Your HTTPS web-app origin(s)	Yes — server refuses to start without it
JWT_SECRET	Random, 32+ characters, kept secret	Yes — minimum length checked at start
DB_USER / DB_PASSWORD	Dedicated least-privilege user, strong password	No — your responsibility
DB_SSL	true if MySQL is reached over a network	No
Demo seed	Never loaded	Yes — seed refuses in production
Rate limiting	At the proxy or platform	No — not implemented in the API

27.2 Creating the first administrator

The website cannot create administrators. In production:

1. Create a normal account on the website (Create account).
2. Promote it in MySQL: `UPDATE users SET role = 'admin' WHERE email = 'person@your-org.example';`
3. Sign in through the administrator sign-in page.

To remove administrator rights: `UPDATE users SET role = 'user' WHERE email = '...';` Admin access stops on the very next request, because the role is re-checked in the database each time.

27.3 Rotating the JWT secret

Changing `JWT_SECRET` invalidates every existing token: all users must sign in again. Do it immediately if the secret may have leaked.

28. Security

Control	Implementation	Status
Password hashing	bcrypt, cost 10; plain passwords never stored or logged	**Implemented**
No password exposure	No endpoint returns `users.password`; user lists select explicit columns	**Implemented**
Authentication	JWT signed with JWT_SECRET, 7-day expiry	**Implemented**
Admin authorization	protectAdmin on all /api/admin and /api/manage routes, with database role re-check	**Implemented**
SQL injection	Every query uses mysql2 placeholders (`?`, `??` for identifiers); no string concatenation of input	**Implemented**
Input validation	Flat JSON bodies only; numeric :id; required fields; database errors mapped to 400	**Implemented**
Error hygiene	Generic 500 message; SQL errors and stack traces only in server logs	**Implemented**
CORS	Allowlist from CLIENT_URL; required in production	**Implemented**
Headers	`X-Powered-By` disabled; no other security headers (HSTS, CSP…)	**Partially Implemented**
Secrets	.env git-ignored; .env.example placeholders only; production startup checks	**Implemented**
Rate limiting	None in the API	**Not implemented**
Public personal data	GET /api/blood, /api/emergency, /api/volunteer, /api/food are public	**Action required**
Token storage	localStorage (readable by any XSS bug)	**Partially Implemented**
Self-delete protection	Web app only; API allows an admin to delete their own account	**Partially Implemented**
Dependencies	npm audit on 26 Oct 2026 reports advisories with fixes available	**Action required**

Table 9 — Security controls

28.1 Dependency audit (26 October 2026)

Package	Project	Severity	Advisory topic (fix available via npm audit fix)
proxy-addr	server	Critical	IP spoofing via IPv4-mapped IPv6 trust subnet (matters when “trust proxy” is configured; it is not set today)
body-parser	server	High	Denial of service via trailing bytes after a compressed body
vite	client	High	Development-server issues (launch-editor, custom middleware, fs.allow); the dev server is not used in production
follow-redirects	client	High	Credential headers on cross-origin redirects (Axios dependency)
source-map-js	client	High	Denial of service through crafted source maps (build tooling)

> Recommended action
>
> Run `npm audit fix` in server/ and client/, then re-run the test suites, build and lint (section 32) before going live. The updates were not applied in this handover so that the documented, tested versions stay unchanged.

29. Monitoring & Maintenance

29.1 What the application provides

- **Health check:** `GET /` returns `{"success": true, ...}` when the API is running. It does not test the database.
- **Logs:** the API writes to standard output/error: startup lines ("Server running", "MySQL Database Connected" or "MySQL Connection Failed") and every server error with a label such as `Create Event Error:`. Capture them with your process manager or platform.
- There is no built-in metrics, alerting, audit log or log rotation (**Future Enhancement**).

29.2 Routine maintenance schedule

Frequency	Task
Daily	Check that the health check answers; review error lines in the API log; confirm the nightly database backup succeeded.
Weekly	Check disk space on the database server; review new admin-visible records for spam or abuse (the public forms have no CAPTCHA).
Monthly	Run `npm audit` and `npm outdated` in server/ and client/; apply security updates, then run the tests, build and lint. Test a backup restore.
Quarterly	Apply operating-system, Node.js and MySQL security updates; review administrator accounts (`SELECT name, email FROM users WHERE role = 'admin'`); review `CLIENT_URL` and other settings.
As needed	Rotate JWT_SECRET or database passwords after staff changes or a suspected leak.

29.3 Database health

- Tables grow without limit (nothing is archived). Watch the size of the MySQL data directory.
- All list endpoints return whole tables; when tables reach thousands of rows, add pagination (section 34) before the admin screens slow down.
- Useful checks: `SELECT COUNT(*) FROM emergencies;` per table, and `SHOW TABLE STATUS;` for sizes.

30. Troubleshooting

Problem	What you see	Possible cause	Solution
Cannot sign in	"Invalid email or password"	Wrong credentials	Check both; ask an administrator to delete and re-create the account (no reset feature)
Admin sign-in refused	"doesn't have administrator access"	Account role is user	Promote with SQL (section 27)
Database connection failed	"MySQL Connection Failed", ECONNREFUSED, requests return 500	MySQL not running / wrong host or port	Start MySQL; check DB_HOST, DB_PORT; restart the API
Access denied	Access denied for user …	Wrong DB_USER / DB_PASSWORD	Fix server/.env; test with mysql -u <user> -p
Unknown database	Unknown database 'lifepro'	Migration not run	npm run sql:migrate
Migration fails	"Migration failed: …"	Privileges, SQL error, tables made by hand	Read the hint; fix and re-run (applied files are skipped)
Seed inserts 0 rows	"0 row(s) inserted"	Demo rows already exist	Expected — the seed never duplicates
API unavailable	"We couldn't reach the LifeLink Pro server"	API stopped or wrong VITE_API_URL	Start the API; check http://localhost:5000/; rebuild with the right VITE_API_URL
CORS error	"blocked by CORS policy" in the browser console	Page origin not in CLIENT_URL (localhost ≠ 127.0.0.1; port 5174)	Add the origin to CLIENT_URL and restart the API
Server stops at start	"Missing environment variables" or JWT_SECRET length message	Incomplete configuration	Complete server/.env (section 23)
Port in use	EADDRINUSE :5000	Another API process still running	Stop the other process
'vite' is not recognized	npm run dev fails in client/	client/node_modules missing	npm ci in client/
404 after refresh (production)	Web server 404 on /events etc.	No SPA fallback	Serve index.html for unknown paths
TLS errors	self-signed certificate…	DB_SSL=true without the provider CA	Set DB_SSL_CA, or DB_SSL=false locally
"Internal server error"	500	Unexpected server/database error	Read the API log for the real message
"Your session has expired"	Redirect to sign-in	Token older than 7 days, account deleted or secret rotated	Sign in again

Table 10 — Troubleshooting guide

31. Backup & Recovery

LifeLink Pro has no built-in backup. All application data is in the MySQL database, so backing up the database (plus the configuration, stored securely and separately) is enough to recover.

What	How	Where to keep it
Database `lifepro`	`mysqldump` nightly (or your provider's automated backups)	Off the database server, encrypted, with retention (e.g. 30 days)
Configuration	Copy of production environment variables	A password manager or secrets store — never with the code
Source code	The delivered project folder or your Git repository	Version control

Example backup (prompts for the password)

```
mysqldump -u <USER> -p --single-transaction --routines lifepro > lifepro_backup_2026-10-26.sql
```

Example restore into an empty database

```
mysql -u <USER> -p -e "CREATE DATABASE IF NOT EXISTS lifepro CHARACTER SET utf8mb4"
mysql -u <USER> -p lifepro < lifepro_backup_2026-10-26.sql
```

1. Stop the API (or put the site in maintenance) so no writes happen during a restore.
2. Restore the latest good backup into an empty database.
3. Run `npm run sql:migrate` (it reports “up to date” if the backup has the latest schema).
4. Start the API and check the health check, sign-in and the admin dashboard.
5. Test a restore at least monthly on a separate server.

> **Deletes are permanent**
>
> Records deleted in the admin area can only be recovered from a backup taken before the deletion.

PART E

Quality, Limitations & Handover

Testing evidence, security checklist, known limitations, future work, developer handover and the final summary.

32. Testing & Validation

33. Security Checklist

34. Known Limitations

35. Future Enhancements

36. Project File Structure

37. Developer Handover Guide

38. Migration History (MongoDB → MySQL)

39. Support & Maintenance

40. Client Handover Checklist

41. Glossary

42. Final System Summary

32. Testing & Validation

Only tests that were actually executed are listed. Dates show when each was run.

32.1 Automated checks (run on 26 October 2026 while preparing this document)

Check	Command	Result
API end-to-end suite (every endpoint, frontend payloads)	`npm run test:api`	78 / 78 passed
Security suite (password exposure, auth, validation, CORS, configuration)	`npm run test:security`	71 / 71 passed
Frontend production build	`npm run build`	Passed — 225 modules; main JS 403.21 kB (123.63 kB gzipped)
Frontend lint	`npm run lint (oxlint)`	Passed — no warnings or errors
Schema migration (existing database)	`npm run sql:migrate`	Passed — up to date
Seed idempotency	`npm run sql:seed`	Passed — 0 rows inserted on re-run
Dependency audit	`npm audit`	Advisories found (section 28)

Both suites create their own test rows and delete them afterwards, and refuse to run when `NODE_ENV=production`.

32.2 Earlier validation recorded in the project history (2–3 October 2026)

Phase	What was run	Result
MongoDB → MySQL migration	Migration and seed on a fresh throwaway database (dropped afterwards); re-runs for idempotency; foreign-key behaviour inside a rolled-back transaction	Passed
Post-migration audit	API suite and security suite; DB_SSL with a local self-signed certificate (correctly rejected)	78/78 and 71/71 passed
Frontend redesign — click-through (real browser, real API and MySQL)	Registration (validation, duplicate email, success), sign-in (wrong password, success, return to page), members-only redirect, member blocked from admin, admin sign-in from both pages, sign-out from four places, session expiry; every service form; create and join event; hospital and ambulance registration with directory search/filter/empty states; admin dashboard totals vs database; search, filters, delete (cancel, Escape, confirm, already-deleted, server down); load-error state; mobile menus	Passed after fixing the defects found
Frontend redesign — responsive	27 routes at 320, 360, 375, 390, 414, 640, 768, 1024, 1280, 1440 and 1920 px: no horizontal page scroll, no element off-screen, no text overflowing buttons, one h1 and a title per page	Passed

32.3 Not tested

- Load, performance and concurrency testing.
- Browsers other than the Chromium-based browser used for testing (Firefox and Safari **Not verified**).
- Screen-reader testing (accessibility was checked through markup and keyboard behaviour only).
- Any production or staging deployment, HTTPS setup or backup restore (**Not verified**).
- There are no automated frontend (UI) tests.

33. Security Checklist

Verified against the code on 26 October 2026. Items marked "Deployment task" depend on how and where the client deploys.

	Item	Status	Evidence / action
☐	HTTPS enabled	**Deployment task**	Not part of the code; required for web app and API
☐	Strong JWT secret	**Deployment task**	Code refuses < 32 characters in production; the value itself is **Not verified**
☐	Database password secured	**Deployment task**	Only in server/.env / environment; use a dedicated user
☑	.env not committed	**Implemented**	.gitignore excludes .env and .env.*; remove the local server/.env before sharing the folder
☑	Password hashes never returned by APIs	**Implemented**	Explicit column lists; covered by security tests
☑	SQL queries parameterized	**Implemented**	pool.execute with placeholders throughout
☑	Admin authorization enabled	**Implemented**	protectAdmin + database role re-check
☑	CORS restricted	**Implemented**	CLIENT_URL allowlist, mandatory in production
☐	Database backups configured	**Deployment task**	Section 31
☑	Production seed data disabled	**Implemented**	seed.js refuses when NODE_ENV=production
☐	Dependencies audited and updated	**Action required**	Advisories open (section 28)
☐	Rate limiting on sign-in / registration	**Not implemented**	Add at proxy or in Express
☐	Public personal-data endpoints reviewed	**Action required**	Section 34
☐	Security headers (HSTS, CSP)	**Not implemented**	Add helmet or proxy headers

34. Known Limitations

These are the current limitations. None is hidden; items marked **Before launch** should be resolved before the system is opened to the public.

Feature / area	Status	Limitation	Priority
Sign-in protection	**Not implemented**	No rate limiting or CAPTCHA on login, registration or public forms	Before launch
Personal data	**Action required**	GET /api/blood, /api/emergency, /api/volunteer, /api/food return names and phones without authentication	Before launch
Dependencies	**Action required**	npm audit advisories with fixes available	Before launch
Pagination	**Not implemented**	Every list returns the whole table; admin screens page in the browser	Recommended
Event participation	**Partially Implemented**	Join is anonymous and repeatable; joined state kept per browser	Recommended
Password policy	**Partially Implemented**	8-character minimum in the form only; API accepts any non-empty password; bcrypt uses the first 72 bytes	Recommended

Feature / area	Status	Limitation	Priority
Input trimming	**Partially Implemented**	Forms trim and reject blank text; the API accepts whitespace-only values from other clients	Recommended
Session security	**Partially Implemented**	JWT in localStorage; no server-side logout or refresh tokens	Recommended
Security headers	**Partially Implemented**	Only X-Powered-By disabled	Recommended
Self-delete	**Partially Implemented**	Only the web app prevents admins deleting their own account	Recommended
UI tests	**Not implemented**	No automated frontend tests	Recommended
Emergency status	**Partially Implemented**	Pending → Accepted → Resolved exists in the schema; nothing changes it	Optional
Emergency ↔ user link	**Partially Implemented**	user_id column and foreign key exist; never set	Optional
Ambulances	**Partially Implemented**	No update of availability, no delete	Optional
Record editing	**Not implemented**	No record can be edited by anyone; only deleted	Optional
Accounts	**Not implemented**	No password reset, e-mail verification or profile editing	Optional
Availability flags	**Partially Implemented**	blood_donors.available and food_donations.available never change; events.status unused	Optional
Notifications	**Not implemented**	No e-mail, SMS or push notifications	Optional
Emergency numbers	—	112 and 108 are India's numbers (client/src/config/services.js)	Optional
Dashboard endpoints	—	/api/admin/stats and /api/manage/dashboard run sequential COUNT queries and are unused by the UI	Optional
Timestamps	—	TIMESTAMP columns end in 2038; switch to DATETIME in a future migration	Optional
Form interaction	—	If the last field's error appears exactly as Submit is clicked, the first click can miss (the form was invalid anyway)	Optional

Table 11 — Known limitations

35. Future Enhancements

> Future Enhancement — Not currently implemented
>
> Everything in this section is a suggestion. None of it exists in the current code.

Enhancement	Benefit	Builds on
Emergency status workflow	Coordinators mark requests Accepted / Resolved	Existing status column
Link requests and registrations to accounts	“My requests” on the dashboard	emergencies.user_id
Event participants table	Reliable counts, leave event, attendee lists	New table + signed-in join
Edit records (admin)	Fix mistakes without deleting	Manage routes
Ambulance availability updates	Up-to-date directory	ambulances.status
Password reset and e-mail verification	Self-service account recovery	E-mail service
Notifications (e-mail / SMS)	Alert coordinators to new SOS requests; contact matching donors	Messaging provider
Server-side pagination and search	Performance at scale	List endpoints
Audit log	Who deleted what and when	New table
Reporting and export (CSV)	Analysis outside the app	Admin screens
Role management	Promote/demote administrators in the app; more roles	users.role
Real-time emergency tracking / dispatch	Live coordination	Requires new architecture
Mobile application	Native experience	Existing REST API
Rate limiting, CAPTCHA, security headers	Abuse protection	Express middleware or proxy

36. Project File Structure

The actual tree as delivered (dependency folders and build output omitted).

```
lifepro/
├── .claude/launch.json             dev-server definitions for the Claude desktop app (optional tooling)
├── .gitignore                      node_modules, .env, .env.*, dist, logs (keeps .env.example)
├── README.md                       developer README (setup, API, troubleshooting)
├── POST_MIGRATION_TODO.md          open follow-ups
├── database/
│   ├── README.md                   migration and seed details
│   ├── migrations/001_initial_schema.sql
│   └── seeds/001_initial_seed.sql  demo data (development only)
├── server/                         Express API
│   ├── server.js                   entry: config checks, CORS, middleware, routes
│   ├── config/                     connectionOptions.js, database.js (mysql2 pool)
│   ├── controllers/                adminController, authController, bloodController,
│   │                               emergencyController, manageController
│   ├── middleware/                 authMiddleware.js, errorHandler.js, validation.js
│   ├── routes/                     admin, ambulance, auth, blood, emergency, event, food,
│   │                               hospital, manage, volunteer (*Routes.js)
│   ├── scripts/                    migrate.js, seed.js
│   ├── tests/                      api.test.mjs (78 checks), security.test.mjs (71 checks)
│   ├── uploads/                    empty, unused
│   ├── .env.example                configuration template (placeholders)
│   └── package.json, package-lock.json
└── client/                         React + Vite web app
    ├── index.html, vite.config.js, .oxlintrc.json, .env.example
    ├── public/favicon.svg
    ├── package.json, package-lock.json
    └── src/
        ├── main.jsx, App.jsx       entry and route table
        ├── pages/                  20 page folders (one per route)
        ├── components/             ui, form, layout, admin, auth, content, directory,
        │                           emergency, events, feedback, services
        ├── context/                AuthProvider.jsx, authContext.js, toastContext.js
        ├── hooks/                  9 hooks (useForm, useAsync, useAuth, useSignOut, ...)
        ├── services/               api.js, lifelink.js
        ├── config/services.js
        ├── utils/                  cx, errors, format, status, validation
        └── styles/                 tokens.css, global.css
```

37. Developer Handover Guide

37.1 First day checklist

1. Install Node.js, npm and MySQL (sections 21–22); `npm ci` in server/ and client/.
2. Create `server/.env` from the example (section 23).
3. `npm run sql:migrate` and `npm run sql:seed` in server/.
4. Start both dev servers (section 25) and sign in with the demo accounts from the seed file.
5. Run `npm run test:api` and `npm run test:security` with the API running; both should pass. Override the admin login with `TEST_ADMIN_EMAIL` / `TEST_ADMIN_PASSWORD` and the API address with `API_URL` if needed.

37.2 Conventions

- **Backend:** CommonJS; one router per `/api/<module>`; always `pool.execute(sql, [params])`; always `sendServerError(res, error)` in catch blocks after `console.error`; responses carry `success` and `message`.
- **Admin endpoints:** add `protectAdmin` (and `validateIdParam` for `:id`). Never select `users.password`.
- **Database:** schema changes only through new migration files; keep NOT NULL aligned with API validation.
- **Frontend:** add API calls to `services/lifelink.js`; use `useForm` + `utils/validation.js` for forms, `useAsync` for lists, the UI kit for buttons, alerts, empty/error states; style with CSS Modules and the variables in `tokens.css`.
- **Admin screens:** a new management screen is mostly one entry in `components/admin/adminResources.jsx` (title, load/remove functions, columns, search fields, filter).
- **Lint/build:** keep `npm run lint` at zero warnings and `npm run build` passing.

37.3 How to add a feature end to end (example: emergency status)

1. Migration `002_...sql` if the schema changes (here it does not: the status column exists).
2. Route `PUT /api/manage/emergencies/:id/status` with protectAdmin and validateIdParam; validate the value against the ENUM; `UPDATE ... WHERE id = ?`.
3. Add checks to `server/tests/api.test.mjs` and, for access control, `security.test.mjs`.
4. Add `manageApi.setStatus` in `lifelink.js` and an action in the emergencies entry of `adminResources.jsx`.
5. Update README.md and this document's API section.

38. Migration History (MongoDB → MySQL)

Earlier versions stored data in MongoDB through Mongoose. The API routes had already been rewritten for MySQL, but the project still carried unused Mongoose models, the `mongoose` dependency and `MONGO_URI`, and had no SQL schema or setup workflow. The project was standardised on MySQL only (October 2026).

```
Before:  React → Express → MongoDB (models, unused) + MySQL
After:   React → Express → mysql2 → MySQL
```

- **Removed:** `server/models/*.js` (8 Mongoose models imported nowhere), the mongoose dependency (with mongodb/bson) and MONGO_URI.
- **Added:** SQL migration and seed files, `npm run sql:migrate` / `sql:seed`, database settings in .env instead of hard-coded credentials.
- **Compatibility fixes:** routes accept the field names the forms send (eventName, hospitalName, driverName, availability); fields the forms send are now stored (blood age; volunteer skills, location, availability; event organiser; hospital address, services, emergency availability; ambulance vehicle number and status; emergency type and description); manage lists return camelCase aliases; delete buttons use `id` instead of MongoDB's `_id`; `events.date` is a real DATE returned as YYYY-MM-DD.
- **Data:** no MongoDB export existed in the project, so no records were transferred. `database/README.md` describes a safe procedure if an old MongoDB instance still holds data.

MongoDB model (collection)	MySQL table	Notable mapping
User (users)	users	role → ENUM; email UNIQUE
BloodDonor (blooddonors)	blood_donors	bloodGroup → blood_group; phone UNIQUE
FoodDonation (fooddonations)	food_donations	donorName → name; foodType → food_type
Volunteer (volunteers)	volunteers	address added
Event (events)	events	eventName → title; date String → DATE
Hospital (hospitals)	hospitals	hospitalName → name; emergencyAvailable → emergency_available
Ambulance (ambulances)	ambulances	driverName → name; vehicleNumber → vehicle_number; availability → status
Emergency (emergencies)	emergencies	userId → user_id (FK); emergencyType → emergency_type

Common to all: `_id` (ObjectId) → `id` (BIGINT AUTO_INCREMENT), `createdAt/updatedAt` → `created_at/updated_at`.

39. Support & Maintenance

The project does not define a support organisation, contact details or service levels; the client should decide these. The following makes support efficient:

39.1 First-line checks (anyone)

1. Is the website loading? Is http(s)://<api>/ answering with the welcome message?
2. Is MySQL running and is there free disk space?
3. What exactly did the user see (message text, page address, time)?

39.2 Information to collect for developers

- Page address, steps to reproduce, expected and actual result, screenshot.
- The matching lines from the API log (they include labels such as `Create Event Error:`).
- Browser and device; whether the problem affects one user or everyone.
- Never send passwords, tokens or .env files in support tickets.

39.3 Handing work to a new team

Give them this document, README.md, database/README.md and POST_MIGRATION_TODO.md, plus the source code and — through a secure channel — the production configuration.

40. Client Handover Checklist

	Deliverable	Location	Status
☑	Source code (frontend and backend)	client/, server/	**Included**
☑	Database schema	database/migrations/001_initial_schema.sql	**Included**
☑	Migration script	server/scripts/migrate.js (npm run sql:migrate)	**Included**
☑	Seed script and demo data	server/scripts/seed.js, database/seeds/	**Included**
☑	Environment templates	server/.env.example, client/.env.example	**Included**
☑	Automated tests	server/tests/ (78 + 71 checks)	**Included**
☑	Developer documentation	README.md, database/README.md	**Included**
☑	Open issues list	POST_MIGRATION_TODO.md	**Included**
☑	API documentation	This document, section 19; README section 13	**Included**
☑	Deployment instructions	This document, sections 26–27	**Included**
☑	Administrator guide	This document, section 10	**Included**
☑	User guide	This document, section 9	**Included**
☑	Backup instructions	This document, section 31	**Included**
☑	Security documentation	This document, sections 28 and 33	**Included**
☐	Remove the local server/.env before sharing	server/.env	**Client action**

	Deliverable	Location	Status
☐	Create a Git repository for the source	—	**Client action**
☐	Resolve the "Before launch" limitations	Section 34	**Client action**
☐	Provision production hosting, HTTPS, MySQL and backups	Sections 26–31	**Client action**
☐	Create the first production administrator	Section 27	**Client action**

41. Glossary

Term	Meaning
API	Application Programming Interface — the set of web addresses the app uses to read and save data.
REST API	An API style where each address represents a kind of data and HTTP methods (GET, POST, PUT, DELETE) say what to do with it.
Frontend	The part people see in the browser (here: the React web app in client/).
Backend	The server program that checks requests and talks to the database (here: Express in server/).
Database	Where data is stored permanently.
MySQL	The database software LifeLink Pro uses.
SQL	The language used to create tables and read or change data in MySQL.
Migration	A versioned SQL file that creates or changes the database structure.
Seed	A script that loads example (demo) data for development and testing.
Authentication	Checking who someone is (signing in with email and password).
Authorization	Checking what a signed-in person may do (for example, only admins may delete).
JWT	JSON Web Token — a signed, time-limited "pass" the server gives after sign-in.
bcrypt	A one-way password-hashing method; the original password cannot be read back.
CORS	Cross-Origin Resource Sharing — a browser rule; the API lists which websites may call it.
CRUD	Create, Read, Update, Delete — the basic data operations. LifeLink Pro supports Create, Read and Delete; Update is limited to joining events.
SPA	Single-Page Application — the browser loads the app once and switches pages without reloading.
Environment variable	A setting given to the server from outside the code (e.g. in server/.env).
HTTPS / TLS	Encrypted web traffic.
Endpoint	One API address plus method, e.g. POST /api/blood/register.
Status code	The number in an HTTP response that says if a request worked (200, 201) or why not (400, 401, 403, 404, 500).

42. Final System Summary

Project	LifeLink Pro — community emergency and social-service platform
Frontend	React 19.2 single-page app, React Router 7, Axios, CSS Modules with design tokens, built with Vite 8 (client/)
Backend	Node.js + Express 5.2 REST API, 35 endpoints, mysql2 connection pool, no ORM (server/)
Database	MySQL 8 (lifepro): 8 business tables + schema_migrations; versioned SQL migration and idempotent demo seed
Authentication	bcrypt-hashed passwords; 7-day JWT; admin routes protected by protectAdmin with a database role re-check; roles user and admin
Major modules	Emergency SOS, blood donor registry, food donation, volunteers, community events, hospital directory, ambulance directory, accounts, admin dashboard and management
Deployment	Static frontend + Node.js API + MySQL behind HTTPS (model in section 26); no production deployment verified
Migration	MongoDB → MySQL completed; Mongoose and MONGO_URI removed
Verified status	API 78/78 and security 71/71 checks pass, build and lint pass (26 October 2026). Functionally complete for create, list and delete.
Before launch	Rate limiting, decision on public personal-data endpoints, dependency updates, production hosting with HTTPS and backups (sections 33–34)

42.1 Status overview

Implemented	Partially Implemented	Future Enhancement	Not verified
• Accounts: register, sign in/out, session expiry • Emergency SOS submission • Blood donor registry • Food donation • Volunteers • Events: list, create, join • Hospital directory and registration • Ambulance directory and registration • Admin dashboard • Admin management with delete (7 types) • Help / FAQ • Responsive, accessible UI • SQL migration and seed • Automated API and security tests	• Ambulance management (view only) • Emergency status workflow • Emergency-to-user link • Event participation tracking • User management (view and delete only) • Password policy and trimming in the API • Security headers • Token storage	• Notifications • Password reset • Record editing • Pagination • Audit log • Reporting / export • Role management • Real-time dispatch • Mobile app • Rate limiting	• Production deployment, HTTPS and backups • Production secrets • Browsers other than Chromium • Performance under load